LLM SOUP TO NUTS - CHAPTER 4 - PRINT EDITION

The math prerequisites for understanding LLMs in-depth

Probability, vectors, forward passes, softmax, gradients, backpropagation, and a real browser-trained next-word model - with every learner-controlled transition expanded into a readable static sequence.

This print edition preserves the worked states. For sliders, animation, answer checking, and training, use the live chapter: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

Generated August 15, 2026

CONTENTS

Chapter map

1. The math prerequisites for understanding LLMs in-depth
2. Start where Chapter 1 left off: how surprised is the model?
3. Representing words as vectors - a list of numbers.
4. Stack the pieces into a tiny network we can inspect.
5. Turn raw scores into probabilities, then calculate cross-entropy loss.
6. When the guess is wrong, which way do we nudge?
7. Flow the blame backward to fix the weights.
8. Train a real next-word model on an entire book.
9. Further reading.

LEARNING BY DOING

The math prerequisites for understanding LLMs in-depth

In this chapter, we will build a tiny neural network, train it start to finish right here in your browser, and watch a connected network of numbers learn to finish sentences.

This is the math heavy chapter. However, every concept earns its keep by solving a problem that matters. **Probability** puts a number on how surprised the model is by the real next word. **Vectors** let it compare words. **Loss** converts a wrong guess into a single number. **Gradients** influence which way to nudge the numbers so the next guess is a little better. We will progressively build the math to gain a full understanding of how LLMs work under the hood. No matter what the architecture of the model, they all are built on the concepts you are going to learn in this chapter. Finally, in the spirit of learning by doing, you will do the work to ensure you understand each of the concepts.

Chapter 3 showed what happens first: text is split into tokens, each token receives an integer ID, and that ID selects a row from the embedding table. Real tokenizers often split words into subword pieces. This browser model deliberately treats each whole word as one token so every number remains easy to inspect. From that simplified starting point, token IDs select embedding rows, those rows flow through the network to produce next-word probabilities, a loss is measured, and the parameters move a little closer to right. A modern LLM uses **far** more tokens and parameters, but the training beats are the same.

The exact path into this chapter's tiny network

Text
`the cat`

↓

Toy word tokens
`["the", "cat"]`

↓

Token IDs
`[2, 3]`

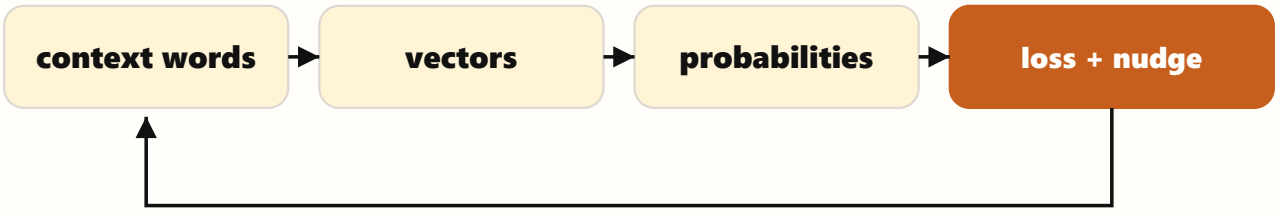
↓

Embedding rows
`E[2], E[3]`

↓

Network
the math we build here

The IDs 2 and 3 are the positions of `the` and `cat` in this toy vocabulary. Production tokenizers use much larger vocabularies and often split a word into several tokens.



repeat for every training example, nudging the numbers a little each time

Guiding question: how can a network of numbers learn to finish your sentence, when nobody wrote the grammar rules by hand? That is the mystery this whole chapter slowly takes apart.

Measure surprise

Turn next-word counts into probability, entropy, cross-entropy, and perplexity.

Compare with vectors

Use embedding rows for the context words, compute a hidden summary, then score each candidate with separate output weights.

Nudge with gradients

Use loss, gradients, gradient descent, softmax, and backprop.

Train the real thing

Assemble a small neural network and teach it to predict the next word.

Before we dive in, a few needed prerequisites are in order. This chapter leans on Chapters 1, 2, and 3, and it builds every piece of math slowly, the moment we actually need it. A little comfort with the following will make the ride smoother - use the links as quick refreshers if anything feels new.

- **Chapter 1.** We reuse next-word counts turned into probability (`P(next | context)`) and the idea of **perplexity** as how "surprised" a model is.
- **Chapter 2.** We reuse the big idea that each word is a short *row of numbers* (a vector), and that a **dot product** compares two of them.
- **Chapter 3.** We reuse the path from text to token IDs to embedding rows. Real tokenizers often produce subword pieces; this chapter deliberately uses one whole word per token so the network

math stays easy to inspect.

- **Multiply, add, compare.** That is the entire math vocabulary on this page. If you can multiply two numbers and add the results, you can follow every formula here.

1. SURPRISE AND THE NEXT WORD

Start where Chapter 1 left off: how surprised is the model?

> *One heads-up before we start.* The other chapters keep the math light on purpose. This one does the opposite: we are going to write down the equations. Do not let that scare you off. Every symbol gets explained in plain words right where it appears, and every formula is just a tidy way of writing a concept you will have already worked out by hand a line above. Where a piece of calculus shows up, I will define it on the spot. Read the words first and the symbols second, and it will all make sense.

In Chapter 1, we learnt how to calculate perplexity from probability. To remind ourselves, perplexity is just a formulation of surprise. Now we will use it again to compare the evidence with a neural network's guesses.

Let's begin with a feeling you already have: *surprise*. Read "the cat ___" and your mind quietly bets on a couple of words - `sat`, maybe `ran`. End the sentence on `sat` and you feel nothing; end it on `helicopter` and you feel something is not right. A language model has that exact reflex, and all of training is just teaching the model to get less surprised and more confident at the words that really come next. So our very first job is to put a number on surprise.

To make this concrete, let's start with a small pile of next-word counts for a specific context. Say that after the phrase `the cat`, we look at our training data and see the word `sat` 8 times, `ran` 3 times, and `slept` once - 12 total. We can express the chance of `sat` coming next as `P(sat | the cat) = 8 / 12 ≈ 0.67` (or 67%). The other two probabilities come from the same count-over-total calculation: `P(ran | the cat) = 3 / 12 = 0.25` (25%), and `P(slept | the cat) = 1 / 12 ≈ 0.083` (about 8%; the panel rounds to whole percentages). The exact fractions add up to `12 / 12 = 1`, or 100%. That vertical bar means "given": we are not asking how common the word `sat` is everywhere, only how common it is *right after* `the cat`. This represents the **true distribution** of our data.

When training a language model, we want its predictions to match this reality as closely as possible. To measure how well it's doing, we rely on three heavily connected concepts from Information Theory. All three answer one fundamental question: *is the model confident, or flailing?* Let's look at them through the lens of "surprise."

To keep the two sides straight, we will call the true distribution in the evidence `p`. In the same order as the words above - `sat`, `ran`, then `slept` - `p = (8/12, 3/12, 1/12) ≈ (0.67, 0.25, 0.083)`. This answers, "What actually happened in the evidence?" We will call the model's current guess `q = (0.55, 0.30, 0.15)`. This answers, "What does the model currently believe?" The two distributions

cover the same three words, but they are supposed to differ before training: `p` is the target and `q` is the imperfect guess. Entropy asks how much uncertainty remains even if we know `p` perfectly. Cross-entropy asks how surprised the model actually is while it uses `q`. Training nudges `q` toward `p`; when they match, cross-entropy reaches the entropy floor.

Entropy is the *inherent uncertainty* of the language itself. Imagine you knew the exact true probabilities (like our 67%, 25%, and 8%). Even with perfect knowledge, you still can't be 100% sure what the next word will be; there is a baseline level of average surprise when the next word is finally revealed. Entropy calculates this absolute floor, measured in bits. It is the *smallest* average surprise anyone could manage on this context. If a context forces exactly one specific next word every single time, the entropy is zero bits—no surprise at all.

Cross-entropy, on the other hand, is the model's *actual* average surprise when it uses its own internal, imperfect probabilities to predict the real next words. If our model incorrectly thinks `slept` is the most likely next word, it will incur a massive "surprise penalty" every time `sat` repeatedly shows up in the training data. The closer the model's predictions match the true distribution, the lower the cross-entropy gets, eventually bottoming out exactly at the true Entropy floor. In machine learning, minimizing cross-entropy is the standard way we teach models to predict text.

As you saw in Chapter 1, **perplexity** rewrites this cross-entropy metric on a much friendlier, intuitive scale. While "bits of surprise" can feel abstract, perplexity translates those bits back into an effective vocabulary size. Mathematically, it is simply the exponentiation of cross-entropy (e.g., $2^{\text{cross-entropy}}$). If a model has a perplexity of 4 on a given text, it means that, on average, the model felt as confused as if it were choosing blindly between 4 equally likely words. A perfect model that knows exactly what comes next has a perplexity of 1. In this chapter, it stays beside cross-entropy as a readable translation of the same surprise, not as a separate training objective.

> *Now feel it for yourself.* The sliders under **Evidence p** are what actually happened - how often each word really followed the context. The sliders under **Model q** are the model's guesses. Drag a guess away from the evidence and watch cross-entropy climb; drag it back and watch the surprise fall toward the entropy floor. That falling number is exactly what training will do for us, automatically, thousands of times over.

Try the controls and motion in the live chapter: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

Evidence `p`: what happened **the cat**

Context

sat	8
ran	3
slept	1

sat



8 / 12 = 67%

ran



3 / 12 = 25%

slept



1 / 12 = 8%

Model $_q$: what it believes

1.19 bits

sat	55
ran	30
slept	15

12
EXAMPLES
counted after this context

1.19 bits
ENTROPY
uncertainty in the evidence

1.24 bits
CROSS-ENTROPY
surprise using the guesses

2.36
PERPLEXITY
effective number of choices

0.05 bits
EXTRA SURPRISE
0 means the guesses match perfectly

sat
BEST GUESS
largest guessed probability

> The guess distribution is very close to the evidence, so cross-entropy is almost as low as the true entropy.

Quick check: this context honestly allows several next words. As the model learns, its cross-entropy falls - how low can it possibly go?

- **All the way to zero**
- **Down to the entropy floor, but no lower**
- **It just keeps rising**

> Answer: Down to the entropy floor, but no lower. Entropy is the unavoidable floor set by the evidence itself; cross-entropy is the model's actual surprise, which sits on or above that floor. Training pushes cross-entropy down toward the entropy floor - it can never dip below it.

Cross-entropy by hand

Now use those same two distributions together. Evidence `p` says how often each word actually happened: `sat 8/12`, `ran 3/12`, `slept 1/12`. Model `q` says what the model currently believes: `sat 0.55`, `ran 0.30`, `slept 0.15`. Cross-entropy goes row by row: `p` supplies how heavily that row counts, while `-log2(q)` supplies how surprised the model is on that row.

```
evidence p: p(sat) = 8/12 ≈ 0.67, p(ran) = 3/12 = 0.25, p(slept) = 1/12 ≈ 0.083
```

```
H(p,q) = p(sat)·[-log2 q(sat)] + p(ran)·[-log2 q(ran)] + p(slept)·[-log2 q(slept)]
```

```
H(p,q) = 0.67·[-log2 0.55] + 0.25·[-log2 0.30] + 0.083·[-log2 0.15]
```

```
≈ 0.58 + 0.43 + 0.23 = 1.24 bits → perplexity = 2^1.24 ≈ 2.36 effective choices
```

The live panel recomputes these as you drag. A perfect model would reach the entropy floor of about 1.19 bits; the gap above it is the model's avoidable surprise, and training exists to shrink it. In our notation, that floor is reached when `q = p`.

The same idea, in symbols

Everything we just discussed has a compact mathematical form. The core building block is the *surprise* of a single word. If our model assigns probability q to a word, its surprise is $-\log_2(q)$ bits of *ideal information*. That value can be fractional; it is not claiming that one real codeword or one game of 20 Questions literally uses a fractional question. Notice how this aligns with our intuition: if the model is highly confident (q is close to 1), the surprise drops to nearly zero. But if the model thinks a word is almost impossible (q is close to 0), the resulting surprise penalty is massive.

To evaluate the model's overall performance, we don't just look at one word; we calculate the *expected* (or average) surprise. We take the surprise of every possible next word and weight it by how often that word **truly** appears in the evidence (p). Adding all these weighted surprises together gives us the Entropy (if using the true probabilities p for the surprise) or Cross-Entropy (if using the model's imperfect guesses q for the surprise):

$$H(p) = -\sum_{w} p(w)\ \log_2 p(w) \qquad H(p,q) = -\sum_{w} p(w)\ \log_2 q(w)$$

$$\text{Perplexity} = 2^{H(p,q)}$$

$p(w)$: The **true distribution** — how often word w actually follows this context in the data.

$q(w)$: The **model's prediction** — the probability the model assigns to word w.

$\sum_w$: The sum over the entire vocabulary. We add up the probability-weighted surprise for every candidate next word.

$\log_2$: Measures the surprise in **bits**, framing the prediction as binary choices. Perplexity turns those abstract bits back into an intuitive effective choice count; perplexity itself has no unit.

This formula was not chosen only because the intuition sounds right. A useful information measure must assign zero surprise to certainty, assign more surprise to less likely events, and *add* the surprises of independent events even though their probabilities multiply. Under mild regularity assumptions, those requirements force a negative logarithm. Averaging that

surprise over what reality actually produces gives $-\sum p \log q$. The optional deep dive below derives this carefully and shows why coding theory, honest probability forecasts, and maximum-likelihood training all arrive at the same expression.

A note on actual training: Later sections (and ML frameworks like PyTorch) switch to using the natural log $\ln$. This measures the exact same surprise, but in *nats* instead of bits. In that environment, the displayed loss is cross-entropy in nats, and perplexity is calculated as e^{loss}.

The intuition is useful, but the formula was not an inspired guess. Several different lines of reasoning constrain us toward the same expression. We will start with the requirements for one event's surprise, then average that surprise over reality, and finally check the result against coding theory and model training.

1. The requirements force a logarithm

Call the surprise of an event with probability q by the name $I(q)$. A sensible information measure should obey a few rules:

- **Certainty means zero surprise:** $I(1) = 0$.
- **Rarer events are more surprising:** as q gets smaller, $I(q)$ gets larger.
- **Small probability changes cause small surprise changes:** $I(q)$ should be continuous rather than jumping unpredictably.
- **Independent surprises add:** if independent events have probabilities q_1 and q_2, their joint probability is $q_1 q_2$, while their information should add.

$$I(q_1 q_2) = I(q_1) + I(q_2)$$

Continuous logarithms are exactly the functions that turn multiplication into addition. Those requirements therefore give $I(q) = -K \log q$, where $K > 0$ chooses the unit. The minus sign makes the surprise non-negative because the logarithm of a probability below 1 is negative.

- Choose base 2 and $K = 1$: surprise is measured in **bits**.
- Choose the natural log and $K = 1$: surprise is measured in **nats**.

That is why each halving of probability adds one bit: $-\log_2(1/2) = 1$, $-\log_2(1/4) = 2$, and $-\log_2(1/8) = 3$.

2. Reality supplies the weights; the model supplies the surprise

Now suppose reality produces outcome i with probability p_i, while the model assigns that outcome probability q_i. Whenever outcome i occurs, the model pays $-\log q_i$ surprise. Reality makes some outcomes occur more often than others, so the average surprise is the expectation under p:

$$H(p, q) = \mathrm{E}_{i \sim p}[-\log q_i] = -\sum_i p_i \log q_i$$

- p_i says how often reality makes us pay for row i.
- q_i determines how expensive or surprising that row is when it occurs.

This is the same division of labor in the worked example above: evidence p supplies the frequency weights, and model belief q supplies each word's surprise. That expected surprise is cross-entropy.

3. The same formula is expected coding cost

Now look past the metaphor of surprise. In an ideal code based on model probability q_i, outcome i has information length $-\log_2 q_i$ bits. Frequent model outcomes get short descriptions; outcomes the model considers rare get long descriptions.

Imagine playing 20 Questions to identify the next word, where every yes/no question costs one bit. Probabilities that are exact powers of two make the relationship especially visible:

- A 50% event needs $-\log_2(0.5) = 1$ bit.
- A 25% event needs $-\log_2(0.25) = 2$ bits.
- A 12.5% event needs $-\log_2(0.125) = 3$ bits.

Real prefix codes use whole-bit codewords, so an individual codeword is not always exactly $-\log_2 q_i$ bits. The source coding theorem says that, over long sequences, good codes can approach this ideal average. If the data really follows p but the code was designed from q, its expected ideal length is $-\sum_i p_i \log_2 q_i$: cross-entropy in bits.

So a bad model wastes bits. If `sat` occurs often under p but the model gives it $q = 0.01$, we repeatedly pay about 6.64 bits for a common event.

4. Why reporting the true probabilities wins

Cross-entropy separates into the unavoidable uncertainty in reality and an extra penalty for using the wrong model:

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$$

$$D_{\mathrm{KL}}(p \| q) = \sum_i p_i \log \frac{p_i}{q_i} \geq 0$$

The KL divergence cannot be negative, and it is zero when $q = p$ (apart from zero-probability edge cases). Therefore $H(p, q) \geq H(p)$: the model cannot beat reality's own entropy floor, and cross-entropy reaches that floor when its probabilities match reality.

In forecasting language, negative log probability is a **strictly proper scoring rule**: in expectation, the best strategy is to report your real probabilities rather than pretending to be more or less confident. That makes cross-entropy useful for learning honest probability distributions, not merely choosing the winning class.

5. Maximum likelihood arrives at the same objective

Suppose the training data contains observed outcomes $x_1, \ldots, x_N$, and a model with parameters θ assigns each one probability $q_\theta(x_n)$. Maximum likelihood asks for parameters that make the entire observed dataset as probable as possible:

$$\max_\theta \prod_{n=1}^{N} q_\theta(x_n)$$

Logs turn the product into a sum. Negating and dividing by N turns maximization into minimization:

$$\min_\theta -\frac{1}{N} \sum_{n=1}^{N} \ln q_\theta(x_n)$$

That is empirical cross-entropy: the average negative log probability assigned to the outcomes that actually occurred. With one-hot targets, each example contributes only $-\ln q_{\text{true}}$, exactly the loss used later in the Softmax section.

6. The historical thread

Ralph Hartley used logarithms in 1928 to measure information through the number of possible messages. In 1948, Claude Shannon gave the modern communication-theory treatment: he formalized entropy's defining properties and connected it to the limits of coding. Cross-entropy is the natural next case: reality follows p, but our code or model uses q.

These viewpoints meet at one formula for a reason. The logarithm satisfies the information rules; the expectation under p measures real average cost; KL divergence guarantees truth is optimal; and maximum likelihood turns observed data into the same training loss.

2. WORDS AS VECTORS

Representing words as vectors - a list of numbers.

Entropy tells us *how wrong* our guess was. But before the model can guess at all, it needs a way to tell words apart - and, more usefully, to notice when two words are *alike*. You already do this without thinking: `cat` and `dog` feel like cousins, while `cat` and `table` feel like strangers. We want to give the model that same gut feeling, using nothing but numbers. Chapter 2's answer was to hand each word a short list of learned numbers, its vector. Stack all those rows into one table and you have an **embedding matrix**: one row per word, and "looking up a word" is simply grabbing its row. The row is not a dictionary definition; it is something that the learning algorithm can tune. Here we shrink each word down to just two numbers, so we can draw it as an arrow on a grid and actually see what the model sees.

Why arrows? Because we can compare two word vectors with one humble operation: the **dot product**. Multiply the matching numbers of two vectors, add the results, done. That single number runs large when two arrows point the same way, near zero when they are unrelated, and negative when they point in opposite directions. **Cosine similarity** is the same idea stripped down to direction alone, and **distance** just asks how far apart the tips sit. Cosine tells us whether these two embedding rows point alike. In the next section, we will reuse the dot product inside the network itself.

A QUICK MAP OF NUMBER ARRANGEMENTS

One number, one row, one table, one batch.

Before we use these words in formulas, let's look at how they are laid out. Each object below is a different **number arrangement**. We describe that arrangement by counting along each axis: how many rows, how many numbers per row, and, when we stack several tables, how many tables. Code libraries call that list of counts the array's **shape**. So `4 x 2` means four rows with two numbers in each row.

Scalar: one number on its own, such as the cross-entropy loss `1.24`. It tells us how wrong one result is.

Vector: one ordered row of numbers, such as the `cat` embedding `[0.70, -0.30]`. Its number arrangement is one axis of length `2` because this teaching vector holds two numbers.

Matrix: vectors stacked into a rectangular table. The four embedding rows for `cat`, `dog`, `mat`, and `rug` form a `4 x 2` matrix: four rows, two numbers per row. The network's layer weights are matrices too.

Why do we need a tensor?

A model rarely processes only one training example at a time. Suppose we send it these two contexts together: `the cat` and `the dog`. Each word already has a two-number embedding:

```
example 1, "the cat" = [[0.20, 0.10], [0.70, −0.30]]
```

```
example 2, "the dog" = [[0.20, 0.10], [0.60, −0.20]]
```

```
batch = [ example 1, example 2 ]
```

Each example is a `2 x 2` matrix: two context words, with two embedding numbers for each word. Stacking the two matrices gives the computer one object with three axes. Machine-learning libraries call that object a **tensor**. Its code-library shape is `2 x 2 x 2`, and now each `2` has a concrete job:

First axis - 2 examples: should I read `the cat` or `the dog`?

Second axis - 2 context positions: should I read the first word or the second word in that example?

Third axis - 2 embedding numbers: which coordinate of that word's vector should I read?

That is the motivation for a tensor: it keeps several related tables in one organized object without losing which example, word position, or vector coordinate each number belongs to. **Tensor** is also the umbrella term used by machine-learning libraries: a scalar is a zero-axis tensor, a vector is a one-axis tensor, a matrix is a two-axis tensor, and this batch is a three-axis tensor.

Try the controls and motion in the live chapter: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

Move two word vectors

similar direction

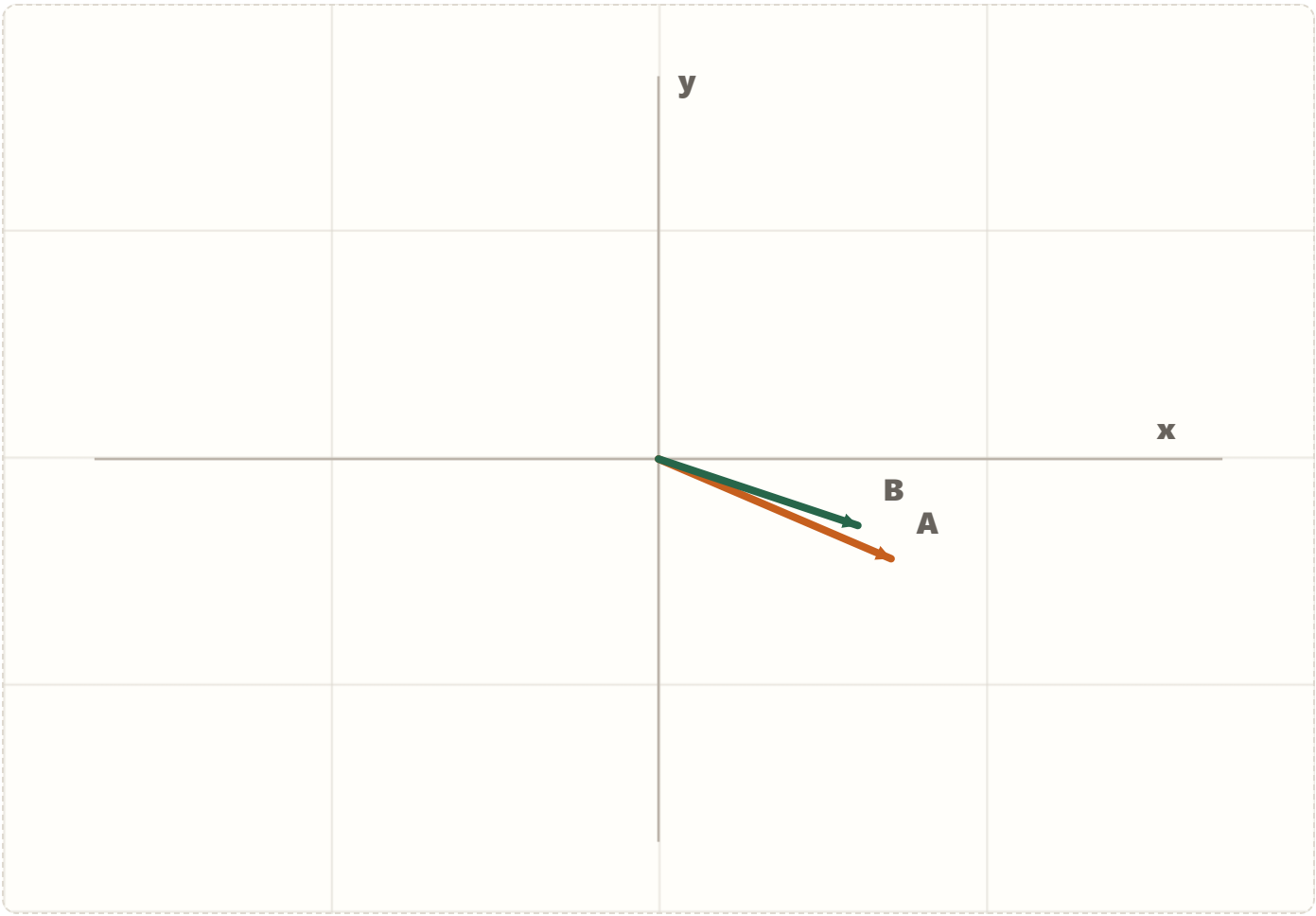
Think of **A** as the vector for `cat` and **B** as the vector for `dog`. Type exact components and watch how similar their directions are. `cat=[0.70,−0.30]`, `dog=[0.60,−0.20]` - the same rows used in the forward-pass panel below.

A x

A y

B x

B y



What the comparison says

dot 0.48

0.480
DOT(A, B)
direction plus length

0.997
COSINE
direction only

0.141
DISTANCE
space between points

4.8 deg
ANGLE
smaller means more aligned

> This panel compares two embedding rows. A high cosine means they point in nearly the same direction; it does not make a next-word prediction. We build that prediction in the next section.

Inspect a row

matrix row

`cat -> [0.70, -0.30]`

nearest row

`dog with cosine 0.997`

lookup idea

`The token ID chooses a row. Training changes the row values.`

One dot product, by hand

Say `cat = [0.7, -0.3]` and `dog = [0.6, -0.2]`. Their similarity score is just multiply-and-add:

```
dot(cat, dog) = 0.7×0.6 + (−0.3)×(−0.2) = 0.42 + 0.06 = 0.48
```

```
cosine = dot ÷ (length cat × length dog) ≈ 0.48 ÷ (0.762 × 0.632) ≈ 0.997
```

A cosine near 1 means `cat` and `dog` are pointing in almost the same direction, so the model can share what it learns about one with the other. That shared strength is the whole reason Chapter 2 moved from exact counts to learned vectors.

The same comparison, in symbols

Why does multiplying and adding numbers measure meaning? For now, imagine a coordinate trying to measure a hidden trait such as "furriness" or "royalty." Chapter 2's important caveat still applies: real dimensions are not individually labelled features; the useful pattern is distributed across the whole vector. This is just for us to get an intuition. When you multiply matching components, you are checking for shared coordinates. If both `cat` and `dog` score high on "furry," multiplying those values produces a large number. Adding these up across all dimensions yields the **dot product**—a raw score of alignment in this intuition.

This mathematical trick wasn't invented for language. In the 1880s, physicists like Josiah Willard Gibbs formulated vector algebra to calculate how much of one physical force aligns with another (like wind pushing against a sail). They proved that the dot product inherently captures the geometric **angle** (θ) between two arrows.

For language models, this geometric angle is crucial. Raw dot products have a flaw: they grow larger simply if a vector is longer (e.g., if a word appears more frequently during training). By dividing the dot product by the lengths of the two vectors, we strip away the magnitude. We are left only with the direction they point. This is **cosine similarity**:

$$a \cdot b = \sum_{i=1}^{n} a_i b_i \qquad \|a\| = \sqrt{a \cdot a} \qquad \cos\theta = \frac{a \cdot b}{\|a\|\,\|b\|}$$

a_i: the i-th coordinate inside vector a; one coordinate has no reliable human label on its own.

n: how many numbers each vector holds (just 2 in this demo, but thousands in an LLM).

$\|a\|$: the magnitude or overall length of the arrow a.

θ: the angle between the two arrows. If they point the exact same way (identical meaning), the angle is 0°, and $\cos(0) = 1$.

With $\text{cat} = [0.7, -0.3]$ and $\text{dog} = [0.6, -0.2]$: $a \cdot b = 0.48$, $\|\text{cat}\| \approx 0.762$, $\|\text{dog}\| \approx 0.632$, so $\cos\theta \approx 0.997$. This cosine tells us that the two embedding rows point in nearly the same direction. That is all we are measuring here. In the next section, we will follow these rows through the network and see how they help it make a prediction.

Quick check: two words are used in almost exactly the same way, so training pushed their vectors to point the same direction. Their dot product is...

Near zero

Large and negative

Large and positive - they point the same way

Answer: Large and positive - they point the same way. A dot product (and the cosine built from it) is large when two vectors point the same way. Words used in the same contexts get trained toward the same direction, so their dot product is large - which is how the model shares what it knows about one word with a similar one.

3. BUILD THE FORWARD PASS

Stack the pieces into a tiny network we can inspect.

We have three ingredients so far - probability, word vectors, and the dot product. In this section, we will assemble them into a neural network to predict the next word. We will produce a raw score for every possible next word. In the next section we will convert the scores into **softmax** by turning them into probabilities. Think of a neural network as a pipeline. The input data (our context words) is the water flowing through the pipes, and the **weights** are the adjustable valves. The water always flows forward, but the valves decide how much pressure makes it to each possible next-word output.

The smallest valve intersection is a **neuron**: it multiplies its inputs by weights, adds a **bias**, and runs the result through an **activation function**. Picture it as a tiny opinion-maker: it listens to a few numbers, weighs each one by how much it trusts that input, adds its own standing belief (the bias), and then decides how strongly to speak up. Why bother with the activation? Because without that little bend - a **ReLU**, a $\tanh$, a sigmoid - a stack of layers can only ever draw a straight line, no matter how tall you build it. The bend or non-linearity is what lets it curve and learn complex patterns. To understand this, consider: stacking two plain multiply-and-add steps just gives you another multiply-and-add. Push x through one layer to get $w_1x + b_1$, feed that into a second, and you land on $w_2(w_1x + b_1) + b_2$ - which tidies up into $(w_2w_1)x + (w_2b_1 + b_2)$, a single weight and a single bias all over again. A hundred such layers collapse back into one straight line. The activation's non-linearity is exactly what breaks that collapse: once each layer can curve, stacking them lets the network trace shapes a straight line never could. Line up several neurons and you have a layer; stack the layers and you have a **multilayer perceptron**, or MLP.

This is the part of the forward pass we will build in this section: look up an **embedding** row for each of the two context words - the two words immediately before the one we are trying to predict - then lay those rows end to end to make a single longer row. That laying-end-to-end move is called **concatenation**, and it is worth knowing by name because you will meet it everywhere; here it turns two 2-number rows into one 4-number input. We then push that input through a hidden layer with an activation, and produce one **logit**, or raw pre-probability score, for each candidate word. Use Back and Next below to inspect every multiply-and-add. Then change the context or activation and replay the same five steps. These weights are fixed and illustrative for now; we will learn to adjust them later.

FIVE FORWARD-PASS STATES

One forward pass, laid out from lookup to logits

This PDF lays out every state; the live chapter lets you move backward, forward, and replay: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

STEP 1 OF 5

Look up and join the two embedding rows

Start with the two context words. Each token ID selects one row from the embedding table; joining those rows gives the four-number input.

Goal of this step

Turn the two context words into the four input numbers that the first hidden layer can use.

E **EMBEDDING TABLE**

A table with one learned row of numbers for every word in this toy vocabulary.

e **JOINED INPUT VECTOR**

The new four-number vector made by placing the two selected embedding rows one after the other.

WORD: THE

Look up the row for “the”

The token ID for “the” points to one row in the embedding table E. We write E_the to mean “the row in E selected for the.”

```
E_the = [0.20, 0.10]
```

coordinate 1	coordinate 2
0.20	0.10

WORD: CAT

Look up the row for “cat”

The token ID for “cat” points to one row in the embedding table E. We write E_cat to mean “the row in E selected for cat.”

```
E_cat = [0.70, -0.30]
```

coordinate 1	coordinate 2
0.70	-0.30

Join the rows; do not add them

Put E_cat after E_the. Every coordinate is kept, in the same order.

```
E_the [0.20, 0.10]
```

+ place after +

```
E_cat [0.70, -0.30]
```

→

```
e = [0.20, 0.10, 0.70, -0.30]
```

What we learned: two 2-number embedding rows became one 4-number input vector e.

STEP 2 OF 5

Compute one hidden neuron's dot product

Follow one hidden neuron closely. Multiply each input by its connecting weight, add the four products, then add the neuron's bias.

Goal of this step: calculate the raw input to h1

Hidden neuron h1 gives each of the four input numbers its own importance, called a weight. Those weights live on the incoming connections shown in the picture. Click another hidden neuron to inspect a different set.

Where the weights and bias for h1 live

Each line from a two-number embedding row to the selected neuron stands for two coordinate connections. Each coordinate connection owns one weight.

Embedding row E_the
`[0.20, 0.10]`
two source coordinates

↓

Weights on the connection to h1
`[1.10, 0.30]`
coordinate 1 uses 1.10; coordinate 2 uses 0.30

↓

hidden neuron h1
receives both weighted coordinates

Embedding row E_cat
`[0.70, −0.30]`
two source coordinates

↓

Weights on the connection to h1
`[0.80, −0.40]`
coordinate 1 uses 0.80; coordinate 2 uses -0.40

↓

hidden neuron h1
receives both weighted coordinates

Inside h1: bias `b = −0.10`

The bias belongs to the neuron itself, not to any one incoming connection.

What gets learned during training?

The connection weights and the neuron bias are trainable parameters. They usually begin as small random values. Training repeatedly adjusts them so useful inputs push each neuron in useful ways. This walkthrough freezes one illustrative set of values so you can inspect the calculation.

e **INPUT VECTOR**

The four numbers joined in Step 1: [0.20, 0.10, 0.70, -0.30].

w **WEIGHT**

A trainable multiplier stored on one coordinate connection. It says how strongly that input coordinate pushes this neuron.

b **BIAS**

A trainable constant stored inside the neuron and added after all incoming contributions. For h1, b = -0.10.

the, coordinate 1

`0.200` × `1.100` = `0.220`

the, coordinate 2

`0.100` × `0.300` = `0.030`

cat, coordinate 1

`0.700` × `0.800` = `0.560`

cat, coordinate 2

`-0.300` × `-0.400` = `0.120`

1. Add the four contributions

```
0.220 + 0.030 + 0.560 + 0.120
```

```
0.930
```

This sum is the dot product e · w.

2. Add the bias

```
0.930 + (-0.10)
```

```
0.830
```

This is the raw input to h1; the activation has not been applied yet.

What we learned: h1 compresses four input numbers into one raw value, 0.830.

STEP 3 OF 5

Bend all hidden values with the activation

The activation bends each raw hidden value. This non-linear step is what prevents several layers from collapsing into one multiply-and-add.

Goal of this step

Pass each raw hidden value through ReLU, the selected activation function. ReLU keeps a positive raw value and replaces a negative raw value with 0.

`activation` **THE BEND BETWEEN LAYERS**

Without this bend, stacking multiply-and-add layers would still behave like one large linear calculation.

HIDDEN NEURON H1

Raw input from the dot product: 0.830

```
max(0, 0.830) = 0.830
```

kept as is

HIDDEN NEURON H2

Raw input from the dot product: -0.480

```
max(0, −0.480) = 0.000
```

clipped to 0

HIDDEN NEURON H3

Raw input from the dot product: 0.280

```
max(0, 0.280) = 0.280
```

kept as is

What we learned: the hidden layer now holds [0.830, 0.000, 0.280]. These three activated numbers form the hidden state sent to the output layer.

STEP 4 OF 5

Turn the hidden state into three logits

Each candidate word has its own output weights and bias. One dot product per candidate produces three raw scores called logits.

Goal of this step

Give each candidate next word one raw score. Inspect one candidate path at a time, then compare the three finished logits in the summary.

Where the output weights and bias for “sat” live

The “sat” scorer receives all three hidden values. Each incoming edge owns one output weight; the scorer itself owns one output bias.

Hidden value h1
`0.830`
computed from this input; not a stored parameter

↓

Output weight on h1 → sat
`w = 0.90`
trainable parameter stored on this edge

↓

“sat” scorer
receives this weighted hidden value

Hidden value h2
`0.000`
computed from this input; not a stored parameter

↓

Output weight on h2 → sat
`w = −0.60`
trainable parameter stored on this edge

↓

“sat” scorer
receives this weighted hidden value

Hidden value h3
`0.280`
computed from this input; not a stored parameter

↓

Output weight on h3 → sat
`w = 0.50`
trainable parameter stored on this edge

↓

"sat" scorer
receives this weighted hidden value

Inside the "sat" scorer: output bias `b = 0.10`

This bias is added after the three weighted hidden contributions.

What is learned, and what is computed?

The three hidden values change with the context, so they are computed signals, not stored parameters. The output weights and output bias are trainable parameters. They usually start as small random values and training adjusts them. This walkthrough freezes one illustrative set.

`h` **HIDDEN STATE**

The three activated values from Step 3: [0.830, 0.000, 0.280].

W_2 **OUTPUT WEIGHTS**

One trainable weight on each hidden-to-candidate edge. The selected "sat" path uses [0.90, -0.60, 0.50].

b_2 **OUTPUT BIAS**

One trainable constant inside each candidate scorer. For "sat", $b_2 = 0.10$.

`logit` **RAW CANDIDATE SCORE**

The weighted hidden-state total plus output bias, before softmax creates probabilities.

WORKED CANDIDATE PATH

sat

Multiply each hidden value by the weight on its edge into the "sat" scorer.

h1 → sat
`0.830` × `0.900` = `0.747`

h2 → sat
`0.000` × `-0.600` = `0.000`

h3 → sat
`0.280` × `0.500` = `0.140`

1. Add the three contributions

```
0.747 + 0.000 + 0.140
```

```
0.887
```

This is the hidden-state dot product for “sat”.

2. Add the output bias

```
0.887 + 0.10
```

```
0.987
```

Final raw logit for “sat”.

candidate	raw logit	what it means now
sat	0.987	**largest raw score**
ran	−0.388	smaller raw score
slept	0.262	smaller raw score

What we learned: the selected “sat” path produces logit 0.987. Across all candidates, “sat” has the largest raw score, 0.987. These scores are still not probabilities.

STEP 5 OF 5

Pause at the logits before softmax

The forward network has produced three legal raw scores, but they are not probabilities yet. The next section shows exactly how softmax converts them.

What is complete

The neural network has finished its raw scoring work. It selected embeddings, built a hidden state, and produced one logit per candidate. Nothing in those four steps has converted the scores into probabilities.

COMPLETED IN STEP 1

Embedding rows selected

```
E_the = [0.20, 0.10]; E_cat = [0.70, −0.30]
```

The two token IDs selected these rows from the embedding table.

COMPLETED IN STEP 1

Joined input created

```
e = [0.20, 0.10, 0.70, −0.30]
```

The rows were placed one after the other to make the network input.

COMPLETED IN STEP 2–3

Hidden state computed

```
raw = [0.830, −0.480, 0.280] → ReLU → h = [0.830, 0.000, 0.280]
```

Dot products and biases produced raw hidden inputs; the activation bent them into the hidden state.

COMPLETED IN STEP 4

Candidate logits produced

```
z = [0.987, −0.388, 0.262]
```

One output-weight path produced one raw score for each candidate word.

candidate	raw logit	what it means now
sat	**0.987**	**largest raw score**
ran	−0.388	smaller raw score
slept	0.262	smaller raw score

Problem 1: raw scores may be negative

`ran logit = −0.388`

A probability cannot be negative, so the logits cannot yet be probabilities.

Problem 2: the scores do not form a whole

`0.987 + (−0.388) + 0.262 = 0.861`

A probability distribution must add to exactly 1. These raw logits do not.

What the largest logit tells us

`sat is largest at 0.987`

This makes "sat" the current leader, but 0.987 does not mean 98.7% confidence.

The next problem: turn scores into probabilities

We need one operation that makes every value positive, makes the full set add to 1, and preserves the ranking so the strongest logit still receives the largest probability.

That operation is Softmax. The next section builds it from these exact three logits; no new network weights are involved in that conversion.

Continue to Softmax and assigning blame

THE SAME HIDDEN-LAYER WORK, ALL AT ONCE

One matrix multiplication, by hand.

Why matrix multiplication appears here

Step 2 followed one hidden neuron. The full hidden layer repeats that same dot product for h1, h2, and h3. Matrix multiplication is the compact way to run all three row-by-column calculations together.

W_1	**3 × 4**	3 hidden-neuron rows; 4 weights in each row
e	**4 × 1**	4 input numbers from "the cat"
$W_1 e$	**3 × 1**	1 weighted total for each hidden neuron

Shape check: (3 × 4)(4 × 1) → (3 × 1). The two inside 4s match, so every four-weight row can pair with the four-number input column.

W_1

one row per hidden neuron

h1	1.10	0.30	0.80	−0.40
h2	−0.60	0.90	−0.50	1.00
h3	0.40	−0.70	0.60	0.50

×

e

joined input for “the cat”

the.x	0.20
the.y	0.10
cat.x	0.70
cat.y	−0.30

=

weighted totals

before bias and activation

h1	0.930
h2	−0.680
h3	0.280

Compute the first output cell: row h1 · column e

Select the h1 weight row and the only input column. Multiply matching positions, add the four products, and place the result in the h1 output cell.

the.x with h1 weight

0.200 × 1.100 = 0.220

the.y with h1 weight

0.100 × 0.300 = 0.030

cat.x with h1 weight

0.700 × 0.800 = 0.560

cat.y with h1 weight

−0.300 × −0.400 = 0.120

Add the four products

```
0.220 + 0.030 + 0.560 + 0.120
```

```
0.930
```

Place this value in output cell h1.

Repeat with row h2

`row h2 · column e`

```
-0.680
```

This fills output cell h2.

Repeat with row h3

`row h3 · column e`

```
0.280
```

This fills output cell h3.

Bias is a separate step after the matrix multiplication

Matrix multiplication produced the three weighted totals. Now add one learned bias to each matching hidden neuron.

```
[0.930, -0.680, 0.280] + [-0.10, 0.20, 0.00] = [0.830, -0.480, 0.280]
```

Only after this bias addition does the activation function bend the three raw hidden values.

What we learned: W_{1e} is not new arithmetic. It repeats the dot product from Step 2 once for each hidden-neuron row, producing [0.930, -0.680, 0.280].

THREE JOBS THAT ARE EASY TO MIX UP

Input rows go in; a hidden state is computed; output weights score candidates.

Input lookup

1

`embedding table E`

```
the → [0.20, 0.10]
```

```
cat → [0.70, -0.30]
```

Token IDs select stored rows that represent the context words entering the network.

STORED LEARNED PARAMETERS

↓

join rows; hidden layer computes

Context summary

2

`hidden state h`

```
h = [0.830, 0.000, 0.280]
```

This vector is computed fresh for the current context “the cat”. It is not a vocabulary row and is not itself a stored parameter.

COMPUTED SIGNAL

↓

score candidate “sat”

Output scoring

3

`candidate scorer for “sat”`

```
W₂ weights = [0.90, −0.60, 0.50]
```

```
b₂ = 0.10 → logit = 0.987
```

These separate output parameters compare h with the candidate “sat” and produce its raw logit.

STORED LEARNED PARAMETERS

Keep the roles separate: E supplies context-token rows; h is the context-dependent result of computation; W_2 and b_2 score candidate next words. In this toy, the input and output parameter sets are not shared.

Some production language models reuse one learned matrix for both input lookup and output scoring. This is called **weight tying**. It is an optional architectural choice, not the rule that makes embeddings work. This browser model keeps the two parameter sets separate so you can inspect each job directly.

The whole forward pass, in symbols

A single neuron takes its inputs x, weights them, adds a bias, and runs the result through an activation f:

$$a = f(w \cdot x + b)$$

Stack those into the tiny next-word network. Look up an embedding row for each of the two context words and join them, pass through one hidden layer with the activation f you picked above, and produce one raw score per vocabulary word. The softmax conversion appears in the next section:

$$e = [\, E_{w_1} \,;\, E_{w_2} \,] \qquad h = f(W_1 e + b_1)$$

$$z = W_2 h + b_2$$

E_w: the embedding row looked up for word w; e: the two rows joined.

W_1, b_1: the hidden layer's weights and bias; f: its activation.

W_2, b_2: the output weights and bias that turn the hidden layer into logits z.

z: one raw logit per candidate word. The next section turns these logits into the model distribution q.

Which f? Whichever one you choose. The walkthrough above starts on ReLU, the backpropagation section stays on ReLU because its slope is a tidy 1 or 0, and the model we train at the end of the chapter uses $\tanh$. Only f changes; every other symbol on this page stays exactly where it is.

4. SOFTMAX AND CROSS-ENTROPY

Turn raw scores into probabilities, then calculate cross-entropy loss.

Once we run the network to get its output, it will not naturally hand us neat percentages for the list of possible words. It will hand us a raw score for every word it could predict - a number called a **logit**. A logit can be 4.2 or -1.1 or anything at all, and that is perfectly legal. The problem is not the minus sign; logits are raw relative scores, not probabilities. They do not have to be positive or add up to 1. If all we wanted was the single winning word, the logits would be enough: choose the largest one. But a raw logit such as `4.2` does not mean 42% confident, so logits do not tell us what share of the model's belief belongs to each candidate. Probabilities make every word share one fixed budget of 1, or 100%. If `sat` receives `0.68`, exactly `0.32` remains for all the other words. That shared scale lets us sample plausible alternatives and gives cross-entropy the probability assigned to the word that actually occurred.

WHAT THIS SECTION FINISHES

Two jobs: create a probability distribution, then calculate cross-entropy loss.

1. **Softmax creates probabilities**

   It turns the three logits into probabilities q that are positive and add to 1.

2. **Cross-entropy calculates loss**

   It uses the probability assigned to the word that actually came next to calculate one loss L.

Softmax is the elegant bridge that gets us from any-old-numbers to a clean distribution. It relies on a two-step trick: first, it *exponentiates* every score. This instantly turns every number (even negative ones) positive, and it stretches the gaps between close numbers so a clear winner emerges. Second, it divides each by the running total so the whole set perfectly sums to 1. **Temperature** is the dial that controls that stretching. Chapter 1 gave you a coarse version of this same choice - *top-pick* (always take the strongest word) versus *sampling* (weighted random). Why does one dial do both? Because temperature acts *before* the exponential: every score is divided by `T` first. Make `T` large and you squash all the scores toward zero, so after exponentiating they land

near `e^0 = 1` - nearly equal, a flat tie where even long-shot words get a real chance. Make `T` small and you stretch the gaps between scores wide open, so the exponential of the biggest one towers over the rest and runs away with almost all the probability. Push `T` all the way down toward zero and the single largest score wins outright - which is exactly Chapter 1's top-pick, now recovered as one end of a continuous dial.

Once Softmax gives us probabilities, we calculate the **cross-entropy loss**. It asks one focused question: how much probability did the model give the word that actually came next? High probability for the true word means low surprise and low loss; tiny probability means high surprise and high loss.

Earlier, in **Cross-entropy by hand**, we averaged surprise over the evidence distribution p: words that occurred more often received more weight. Training presents the same idea one observed example at a time. In this example, exactly one word actually came next: `sat`. We record that fact with the target vector $y = [1, 0, 0]$, in the same `sat, ran, slept` order as the probabilities q.

The earlier worked example used $\log_2$, so its surprise was measured in bits. Training code conventionally uses the natural log $\ln$, so this loss is measured in nats. The reasoning is identical; only the unit changes. Start from the full one-example cross-entropy sum, then substitute that target vector:

$$L = -\sum_i y_i \ln q_i$$

$$L = -(1 \ln q_{\text{sat}} + 0 \ln q_{\text{ran}} + 0 \ln q_{\text{slept}})$$

$$L = -\ln q_{\text{sat}} = -\ln q_{\text{true}}$$

The two zero-weighted terms disappear, leaving only the surprise of the word that actually occurred. Averaging this one-example loss across the training data recovers the distribution-level cross-entropy from the earlier worked example: frequent outcomes contribute the same term more often.

With the default probabilities below, the true word `sat` receives $q_{\text{sat}} \approx 0.681$. The live calculation keeps the unrounded probability 0.681487, so its loss is $-\ln(0.681487) \approx 0.383$ nats. If the model gave the true word only 0.10, the loss would jump to $-\ln(0.10) \approx 2.303$ nats. Cross-entropy therefore gives the model one scalar score that clearly distinguishes a confident correct guess from a poor one.

This section completes both the probability distribution q and the loss L. The next section introduces gradients. Backpropagation later differentiates this loss with respect to each logit and carries those derivatives backward; it does not introduce or finish cross-entropy itself.

Choose the example, then walk through it. The default logits, probabilities, and loss are populated because they are the evidence this exercise starts from. Change any score, temperature, or true word below; the bars and every walkthrough step update together.

1. Set the raw word scores

temp 1.00

Context: `the cat ___`. These are the model's raw scores for three candidate next words.

sat	1.70
ran	0.80
slept	−1.10
Temp	1.00
True next word	

2. Read the probabilities and calculate cross-entropy loss

cross-entropy 0.383 nats

sat

68%

ran

28%

slept

4%

The true word is “sat”. Softmax assigns it 68%, so cross-entropy loss is 0.383 nats. Drag a score or temperature and replay the four steps to see where that number came from.

FOUR SOFTMAX STATES

Temperature, stabilization, exponentiation, normalization, and loss

This PDF lays out every state; the live chapter lets you move backward, forward, and replay: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

STEP 1 OF 4

Scale the logits with temperature

Divide every raw logit by the same temperature. This changes how far apart the scores feel without changing their order.

What temperature does

For each candidate, compute scaled score s = z ÷ T. Here T = 1.00. The same divisor applies to every logit, so their ranking is unchanged.

candidate	raw logit z	z ÷ 1.00
sat	1.700	1.700
ran	0.800	0.800
slept	−1.100	−1.100

At T = 1, the scaled scores equal the original logits.

STEP 2 OF 4

Subtract the largest scaled score

Shift every scaled score by the same amount. Their differences stay identical, but the largest shifted score becomes 0.

Why subtract the maximum

The largest scaled score is m = 1.700. Subtract m from every scaled score. The winner becomes 0 and every other value becomes non-positive, which keeps exponentials in a safe numeric range.

candidate	scaled score s	s - m
sat	1.700	0.000
ran	0.800	−0.900
slept	−1.100	−2.800

Gap check

`1.700 − 0.800 = 0.900; after shifting: 0.000 − −0.900`

`0.900`

The gap is unchanged, so Softmax probabilities will be unchanged.

STEP 3 OF 4

Exponentiate the shifted scores

Exponentials turn every shifted score into a positive amount. Larger shifted scores receive larger amounts.

Turn scores into positive amounts

Apply the exponential to each shifted score. Every amount is now positive, and the largest shifted score 0 becomes exactly $e^0 = 1$.

candidate	shifted score	exp(shifted)
sat	0.000	1.000
ran	−0.900	0.407
slept	−2.800	0.061

Add the positive amounts

```
1.000 + 0.407 + 0.061
```

```
1.467
```

This total becomes the denominator for every probability.

STEP 4 OF 4

Normalize, then measure the true word

Divide each positive amount by their total to get probabilities, then read the true word's negative log probability as cross-entropy loss.

Make a probability distribution

Divide every positive amount by their shared total. The results are positive, preserve the logit ranking, and add to 1.

candidate	positive amount	probability q
sat	1.000	0.681
ran	0.407	0.277
slept	0.061	0.041

Probability check

```
0.681 + 0.277 + 0.041
```

```
1.000
```

The probabilities add to 1 (up to displayed rounding).

Cross-entropy reads the true word: “sat”

Softmax gives “sat” probability 0.681. Loss asks how surprising that true outcome was.

```
L = -ln(q_true) = -ln(0.681487) = 0.383 nats
```

What we learned: logits are now a probability distribution, and the true word's probability has become one complete cross-entropy loss number. Backpropagation later calculates the derivative of this loss; it does not replace or finish the loss calculation.

Stable Softmax and cross-entropy, by hand

Take the three raw scores above - `sat 1.70, ran 0.80, slept -1.10` at temperature 1. Subtract the largest score, exponentiate, then divide by the total so the results sum to 1. The subtraction keeps the exponentials numerically safe:

$\max(z) = 1.70 \Rightarrow z - \max(z) = [0, -0.90, -2.80]$

$e^{0} = 1.000,\ e^{-0.90} \approx 0.407,\ e^{-2.80} \approx 0.061 \quad \text{(total: 1.468)}$

softmax: sat $= \frac{1.000}{1.468} \approx 0.681,$ ran $= \frac{0.407}{1.468} \approx 0.277,$ slept $= \frac{0.061}{1.468} \approx 0.041$

if **sat** is true: loss $= -\ln(0.681487) \approx 0.383$ nats

This fixed by-hand calculation uses $T = 1$. The live panel and walkthrough above recompute the scaled scores, stable exponentials, probabilities, and cross-entropy loss at every temperature. At Step 4, cross-entropy always reads the probability of whichever word you selected as true. The only calculation left for Backpropagation is the derivative of this completed loss, not the loss itself.

Units note: a *nat* is just a unit of surprise - the very same idea as a *bit*, only measured with the natural log ($\ln$, base e) instead of base 2, so 1 nat ≈ 1.44 bits. We counted surprise in bits earlier, using $\log_2$; the live loss here uses the natural log $\ln$, so it is measured in *nats* - the exact same concept, which is why perplexity later reads as e^{loss} instead of 2^{loss}.

Stable Softmax and cross-entropy, in symbols

These equations are just the four walkthrough steps above written as shorthand; nothing new is being introduced. Start with original logit z_i for candidate i. Temperature turns it into scaled score $s_i = z_i/T$: a lower T spreads the scores apart, a higher T pulls them together, and their ranking stays unchanged because every logit is divided by the same positive number.

$$s_i = \frac{z_i}{T} \qquad m = \max_j s_j$$

Next choose m, the largest scaled score, and subtract it from every score. The largest shifted score becomes 0 and all the others become non-positive, so no exponential can grow beyond $e^0 = 1$. This is the numerically stable form software uses.

$$q_i = \frac{e^{s_i - m}}{\sum_j e^{s_j - m}}$$

Why is that subtraction allowed? It multiplies every exponential by the same factor e^{-m}. The factor appears in both numerator and denominator and cancels, so the probabilities are exactly the same as ordinary Softmax:

$$\frac{e^{s_i - m}}{\sum_j e^{s_j - m}} = \frac{e^{-m} e^{s_i}}{e^{-m} \sum_j e^{s_j}} = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

Exponentiation makes every amount positive. Dividing by their shared total gives probabilities q_i that sum to 1 while preserving the score ranking. Softmax has now completed its job: the raw scores have become one model distribution q.

Cross-entropy now calculates the loss from that distribution. The target vector y has a 1 at the word that actually came next and 0 everywhere else, so only the true word's term survives. This is the one-example version of the distribution-level cross-entropy from the opening section:

$$L = -\sum_i y_i \ln q_i = -\ln q_{\text{true}}$$

i: the candidate we are currently talking about; j: the index used to scan all candidates in a maximum or sum.

z_i: the original raw logit for candidate i; it can be any real number.

T: temperature; $T = 1$ leaves logits unchanged, lower sharpens, and higher flattens.

$s_i = z_i / T$: candidate i's temperature-scaled score.

m: the largest scaled score, subtracted from every score only for numerical safety.

q_i: the resulting probability for candidate i; every q_i is positive and $\sum_i q_i = 1$.

y_i: the target for candidate i, 1 for the word that actually came next and 0 for every other word.

L: the completed cross-entropy loss in nats, the negative natural log of the true word's probability.

Read the final line in plain words: look up how much probability the model gave the true word, take its negative natural log, and call that one scalar L. High true-word probability gives low loss; low true-word probability gives high loss. No trainable parameter changes during these equations. This is still the forward calculation.

Complete here: logits became probabilities q, and q became cross-entropy loss L. **Next question:** after gradients are introduced, Backpropagation asks how moving each logit would change this already-known loss.

Quick check: you feed three raw scores (logits) into softmax. What is guaranteed about the three numbers that come out?

- They all come out equal
- **Each is between 0 and 1, and together they sum to 1**
- They stay any value, positive or negative

> Answer: Each is between 0 and 1, and together they sum to 1. Softmax exponentiates then divides by the total, so every output lands between 0 and 1 and the whole set sums to exactly 1 - a proper probability distribution. It does not make them equal; a bigger logit still earns a bigger probability.

Quick check: `sat` really came next. Which model guess has the lower cross-entropy loss?

- `q(sat) = 0.80`
- `q(sat) = 0.20`
- They have the same loss

> Answer: q(sat) = 0.80. For one observed next word, cross-entropy is -ln(q_true). Giving the true word more probability makes -ln(q_true) smaller, so q_sat = 0.80 has lower loss than q_sat = 0.20.

WHAT LEAVES THIS SECTION

A probability distribution q and one cross-entropy loss L.

For the default example, $q \approx [0.681, 0.277, 0.041]$, `sat` is true, and the live unrounded calculation gives $L \approx 0.383$ nats. That is a complete forward calculation. The next section teaches how a gradient describes the direction of change; Backpropagation later uses that idea to calculate how every parameter affected this same L.

MIDPOINT: THE GUESS IS COMPLETE

Part A made a guess. Part B will make it learn.

We now have the full forward story: token IDs selected embedding rows, the network produced logits, Softmax made probabilities, and cross-entropy calculated one loss. Nothing has learned yet. The next two sections calculate which way every trainable number should move, then apply those moves.

5. GRADIENTS, GRADIENT DESCENT, AND LOSS

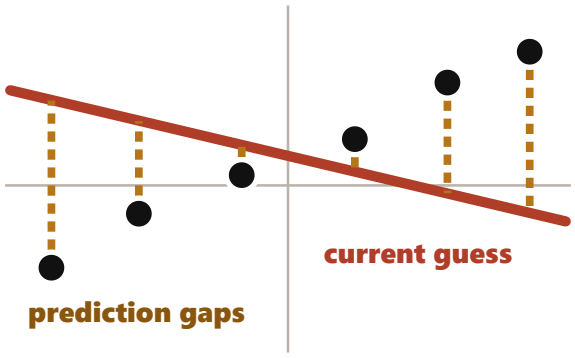
When the guess is wrong, which way do we nudge?

We can now measure surprise and compare words - but measuring a mistake is not the same as fixing it. Here is the question that actually does the learning: when a guess is wrong, which way, and how far, should we move the numbers? Let's start simple, we will shrink the model to the simplest thing that can possibly learn - a straight line with two knobs, a `slope` and an `intercept` - and ask it to pass through a cloud of dots. The **mean squared error (MSE) loss** is just the average squared gap between the line and the dots: one number that answers "how wrong are we right now?". (Our word model will later swap in cross-entropy loss, but the nudging idea is exactly the same.) MSE and cross-entropy are different objectives on different scales, so their raw loss numbers should not be compared with each other.

SEE THE TWO KNOBS BEFORE WE CALCULATE THEM

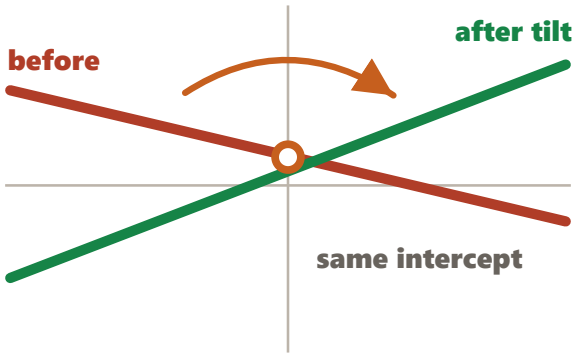
Fitting the line means rotating it and lifting or lowering it until the gaps shrink.

The six dots below are the same six points used by the live lab. The coral line starts at the lab's default `slope = -0.40` and `intercept = 0.20`.



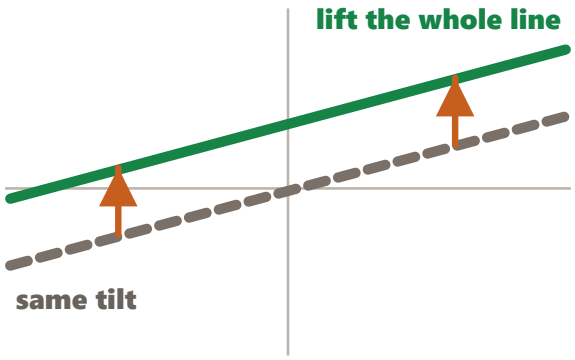
1. Loss measures the gaps

At each dot, the model predicts the height of the line. MSE squares the six vertical gaps and averages them. Big gaps mean big loss.



2. Slope rotates the line

Turn the `slope` knob and the line changes tilt. Points on opposite sides of the centre move in opposite vertical directions.



3. Intercept shifts the line

Turn the `intercept` knob and every prediction moves by the same amount. The line rises or falls without rotating.

Gradient descent does not move the line directly. It measures which way each knob should move to shrink those gaps, then updates the slope and intercept a little at a time.

1

Loss measures the mistake

MSE compresses every gap between the line and the dots into one number: how wrong are we right now?

2

The gradient calculates direction

Imagine standing blindfolded on a hilly valley. For each knob, the gradient asks: if I jiggle this knob, does loss climb or fall, and how steeply? It points uphill, toward steepest increase.

3

Gradient descent applies the update

To go downhill, subtract the gradient. Repeat the measurement and step, using the learning rate to control whether you crawl steadily or leap past the bottom.

FOUR GRADIENT-DESCENT STATES

Measure the loss, find the gradient, plan the update, check the result

This PDF lays out every state; the live chapter lets you move backward, forward, and replay: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

---

STEP 1 OF 4

Measure the current mistake

Start from the line's current slope and intercept, then compress all six point errors into one MSE loss.

What is being measured?

The current line is $\hat{y} = -0.40x + 0.20$. For each of the 6 dots, subtract the true y from the line's prediction, square that error, then average.

Add squared errors

```
1.228 + 0.375 + 0.045 + 0.046 + 0.530 + 1.115
```

```
3.338
```

Every gap contributes positively after squaring.

Average the total

```
3.3381 ÷ 6
```

```
0.556
```

This scalar MSE is the mistake we want to lower.

What we learned: the current slope and intercept produce MSE 0.5563. Loss measures the mistake; it does not yet say how either knob should move.

STEP 2 OF 4

Calculate one gradient per knob

Average the six point contributions to learn which uphill direction belongs to the slope and which belongs to the intercept.

Give each knob its own slope

The slope gradient weights each error by x because changing slope affects distant x-values more. The intercept gradient uses each error directly because changing intercept lifts or lowers the whole line equally.

Slope gradient

`(2 ÷ 6) × −2.9121`

`−0.971`

∂L/∂slope: the uphill direction for the line's tilt.

Intercept gradient

`(2 ÷ 6) × −0.0660`

`−0.022`

∂L/∂intercept: the uphill direction for the line's height.

What we learned: the gradient vector is [-0.971, -0.022]. Each entry belongs to one trainable knob.

STEP 3 OF 4

Plan the opposite, downhill move

Multiply each gradient by the learning rate and subtract it from the matching knob. This previews the update without applying it.

Subtract, because the gradient points uphill

The learning rate η = 0.08 scales both moves. A negative gradient makes subtraction increase that knob; a positive gradient makes subtraction decrease it.

Plan new slope

$-0.40 - 0.08 \times (-0.971)$

-0.322

The slider stores two decimals, so applying the step uses -0.32.

Plan new intercept

$0.20 - 0.08 \times (-0.022)$

0.202

The slider stores two decimals, so applying the step uses 0.20.

What we learned: gradient descent applies the gradients; backpropagation or calculus must calculate them first.

STEP 4 OF 4

Check the predicted result

Recompute MSE with the planned slider values. A useful step lowers loss; an oversized step can raise it.

Did the planned move help?

Use the exact two-decimal values that the sliders will store, draw the planned line, and recompute the same MSE measurement.

BEFORE

`ŷ = −0.40x + 0.20`

MSE 0.5563

Current slider values.

AFTER ONE STEP

`ŷ = −0.32x + 0.20`

MSE 0.4814

Lower loss: this step heads downhill.

Recompute MSE

`0.5563 → 0.4814`

`0.481`

The change is a decrease of 0.0749.

What we learned: this learning rate produces a useful downhill move. Press Step descent below to apply it.

Loss, gradient, and the update rule, in symbols

The loss is the average squared gap between the line's prediction $\hat{y}$ and the true value y. To fix a bad prediction, we need to know how tweaking our knobs affects that loss. In machine learning, we group all adjustable knobs (like our slope and our intercept) under the universal label of **weights**, represented by the letter w. Think of w as a generic blank space: the update rule at the bottom is a template. You run the template once substituting w for the slope, and then you run it again substituting w for the intercept.

The **gradient** $\frac{\partial L}{\partial w}$ reads as "how fast does the loss L change if I nudge just this one specific weight w, holding the rest perfectly still." That is all the partial derivative ∂ symbol means: isolating one knob at a time.

Calculus Refreshers: *If the ∂ symbol looks unfamiliar, Wikipedia has excellent, visual primers on Partial derivatives and the Chain Rule.*

How do we calculate that exact gradient number? We use the Chain Rule. For one data point, the 2 from the squared error drops down to the front, and then we multiply by the inner variable attached to our knob. Because our prediction is $(\text{slope} \cdot x + \text{intercept})$, the variable attached to the slope is x. The variable attached to the intercept is just 1. Those are the per-point contributions. Because the loss above averages over N points, the full gradient adds all N contributions and divides by N too.

$$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \qquad (\text{where } \hat{y} = \text{slope} \cdot x + \text{intercept})$$

$$\frac{\partial L}{\partial\, \text{slope}} = \frac{2}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i \qquad \frac{\partial L}{\partial\, \text{intercept}} = \frac{2}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)$$

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

$\hat{y}$: the line's prediction; y: the true value in the data.

N: how many points we average the loss over.

w: a generic "weight"—a placeholder. You swap in either "slope" or "intercept" when applying the math.

$\frac{\partial L}{\partial w}$: the gradient, representing the uphill slope of the loss for that specific weight w.

η (eta): the **learning rate** or step size (the same slider used in the lab).

$\leftarrow$: "becomes" (we replace the old weight with its newly nudged value).

Two derivatives here become millions of derivatives later

This line has two trainable knobs, so its gradient is the two-number vector below. A neural network follows the identical rule; its gradient vector simply has one entry for every embedding value, weight, and bias.

$\nabla L_{\text{line}} = [\partial L/\partial \text{slope},\ \partial L/\partial \text{intercept}]$

$\nabla L_{\text{network}} = [\partial L/\partial E_{1,1},\ \ldots,\ \partial L/\partial W_{2,1,1},\ \ldots,\ \partial L/\partial b_{2,V}]$

Backpropagation's job is to calculate that longer vector efficiently. Gradient descent's job stays separate: it subtracts the completed vector from the parameters.

Walking through one step for the slope: This is the $N = 1$ special case, so the average contains one term and $1/N = 1$. For a single data point $(x{=}1,\ y{=}1)$ and a starting $\text{slope}{=}0$, our prediction is $\hat{y} = 0$. The error is $\hat{y} - y = -1$.

The gradient for the slope is $2(-1)(1) = -2$. Now we use the update rule template, swapping w for the slope:

$\text{slope} \leftarrow 0 - 0.1(-2) = +0.2$. Subtracting the gradient ensures we step *downhill*, and η (set to 0.1 here) scales how big that step is.

Now make it move. That is the whole recipe - a loss, a gradient, and the one update rule. The exercise just below lets you run it yourself: drag the `slope`, `intercept`, and `step size`, press *Step descent*, and watch this exact loss, gradient, and update step recompute live on real numbers, one nudge at a time. With the math fresh in mind, the numbers in the lab should now read like old friends.

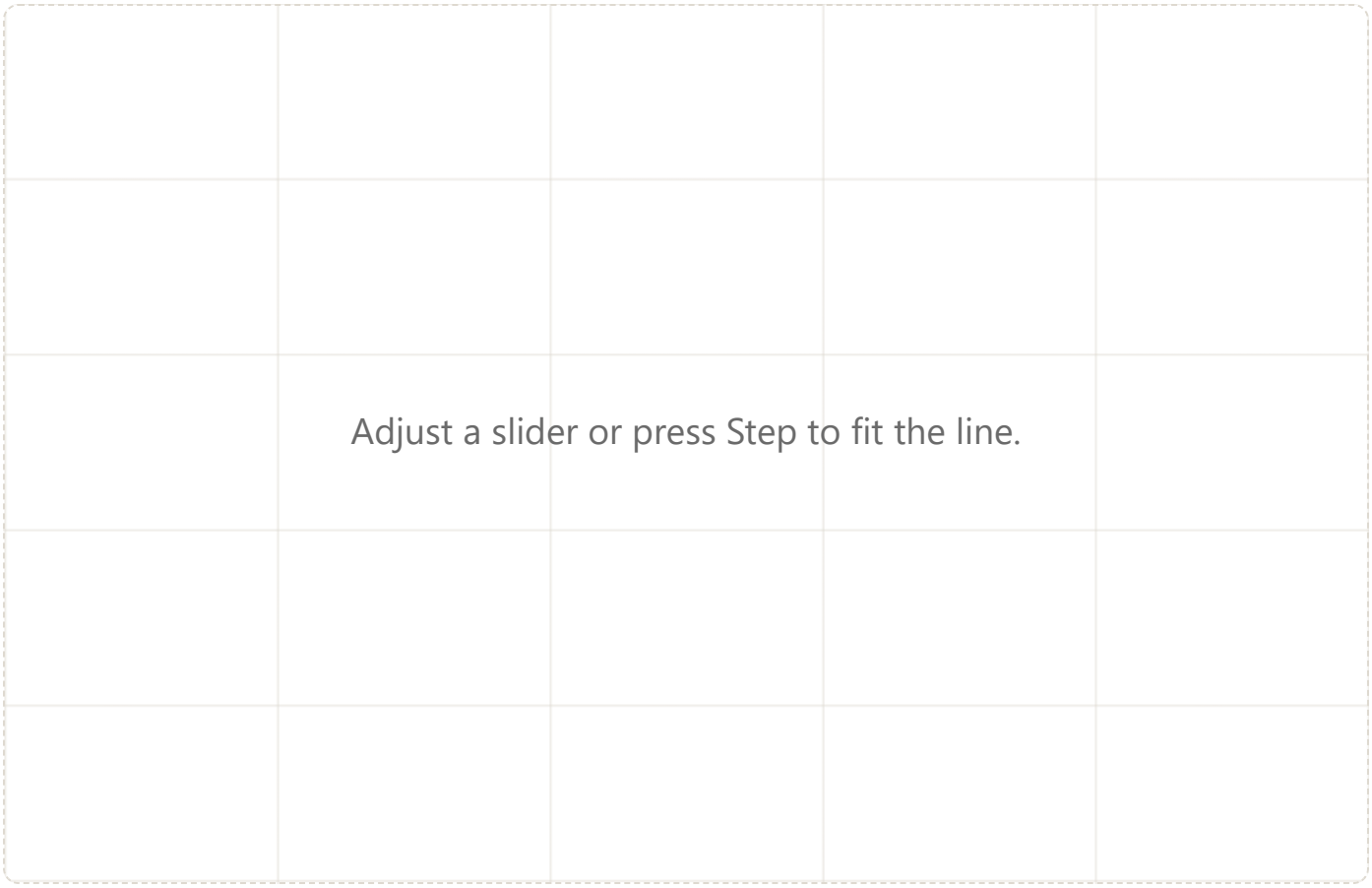
Fit a line by nudging parameters

press Step

Slope	`-0.40`
Intercept	`0.20`
Step size	`0.08`

That **step size** (`0.08`) is the **learning rate**: each step moves a knob by this fraction of its gradient. Small keeps training stable; larger takes bigger, riskier leaps.

Auto train stops when the fit settles, diverges, reaches 80 steps, or runs for 10 seconds. Press Stop at any time.



The sloped line is the model's current guess: its **tilt** is the `slope` and its **height** where it crosses the centre (marked *intercept*) is the `intercept`. Each dot is a data point, and the short vertical gaps between the line and the dots are the **errors** that the loss squares and averages.

The exact math, live

recomputed each step

Move the slope, intercept, or step-size slider — or press Step — to fit the line and watch each knob move.

Quick check: the gradient of the loss points the way that makes the loss *larger*. To make the loss smaller, gradient descent steps...

- **The opposite way - it subtracts the gradient**
- **Along the gradient, the same way**
- **In a random direction each time**

Answer: The opposite way - it subtracts the gradient. The gradient points uphill (toward larger loss), so gradient descent subtracts it - a step in the opposite, downhill direction. That one subtraction is repeated for every weight in the network.

Quick check: you set the learning rate `eta` very large. What is the most likely result during training?

- **The loss reaches its minimum in a single step**
- **Nothing changes; the rate only affects how weights are stored**
- **The updates overshoot the minimum, so the loss bounces or diverges instead of settling**

Answer: The updates overshoot the minimum, so the loss bounces or diverges instead of settling. Each step moves the weights by eta times the gradient. Too large a step jumps past the bottom of the loss curve, so the loss oscillates or blows up instead of converging.

6. BACKPROPAGATION

Flow the blame backward to fix the weights.

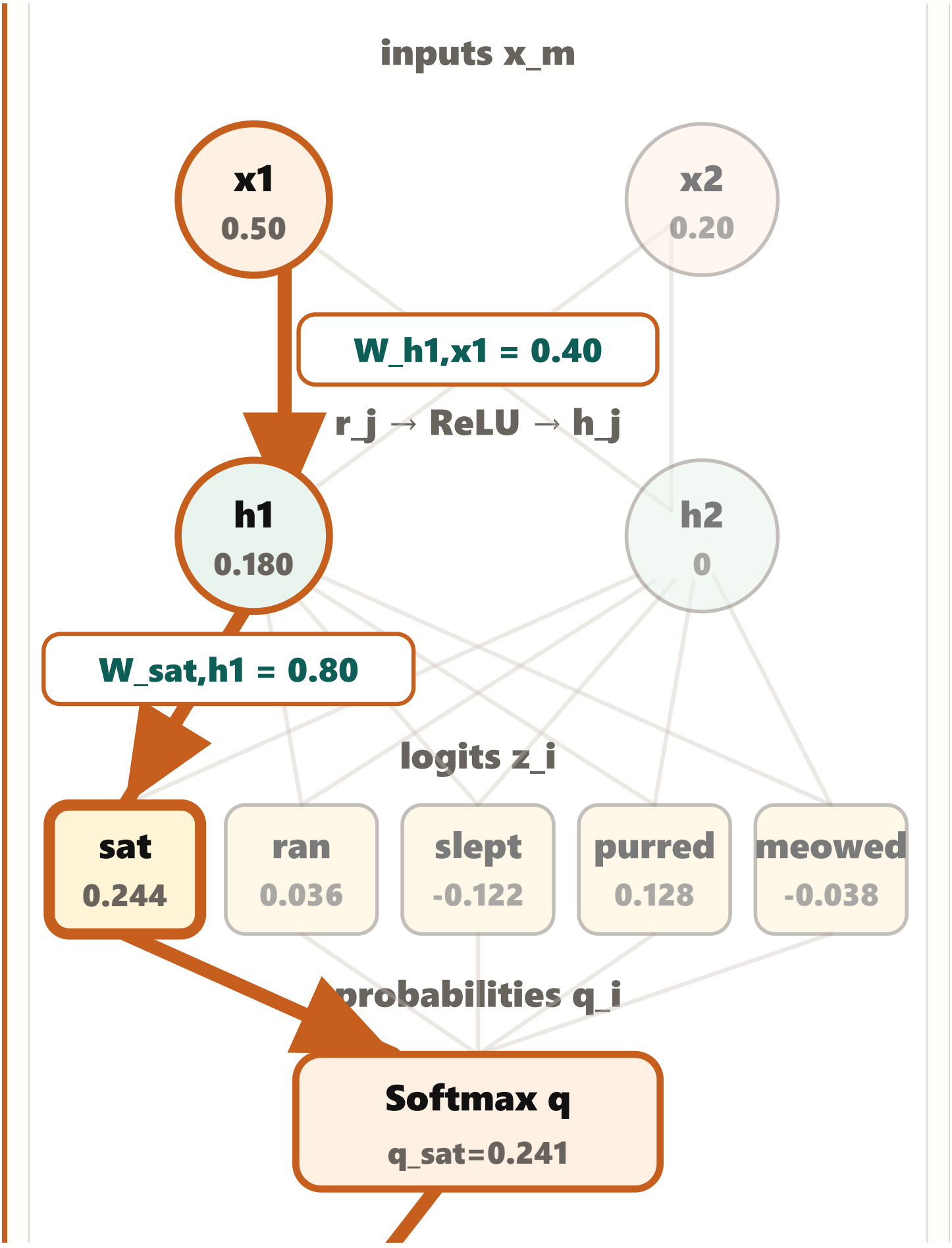
We know the loss, and the preceding gradients section showed what a gradient means. But our tiny network has many knobs buried across two layers. Imagine a factory assembly line: the boss at the end yells, "This car is ruined!" The final assembler replies, "The chassis arrived crooked from the previous station." To fix the factory, we pass responsibility backward through every station in proportion to how strongly that station affected the result.

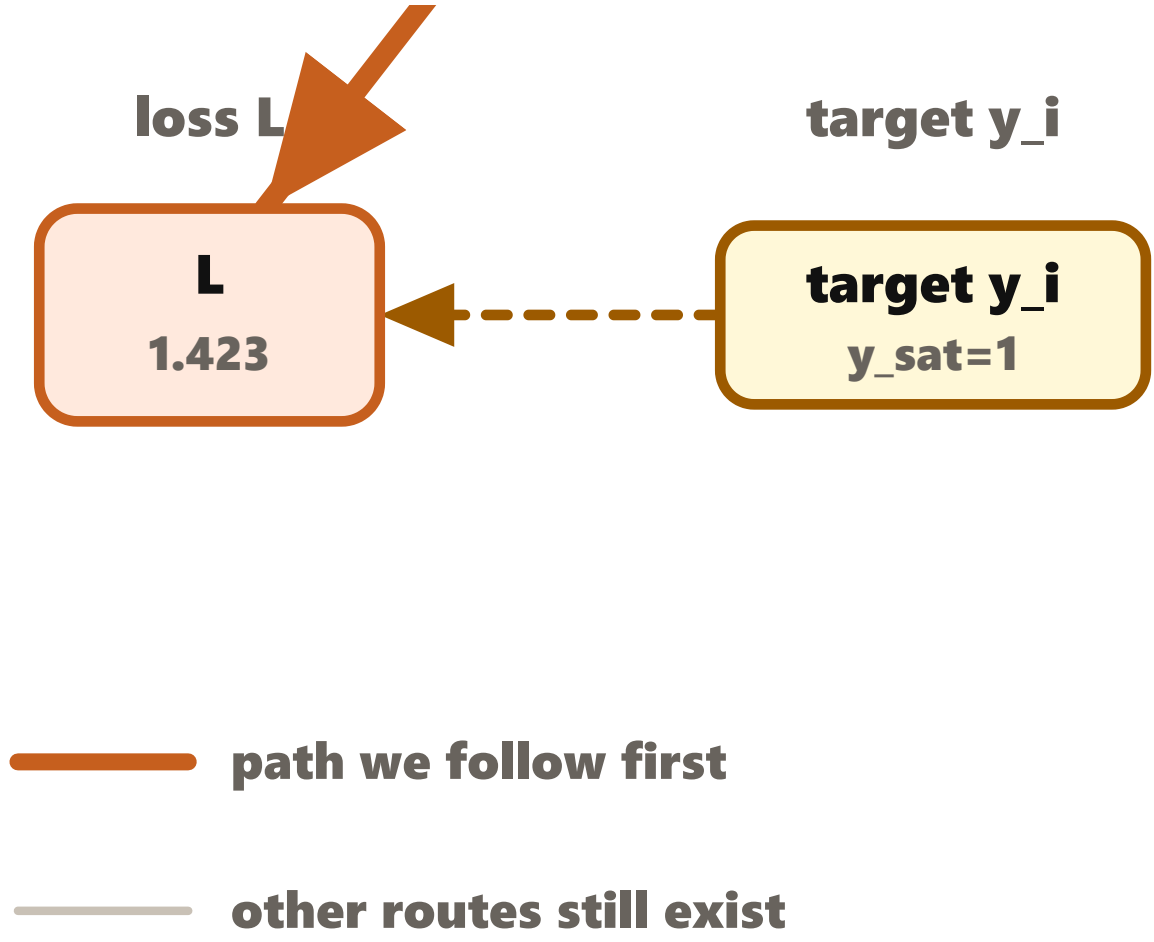
That backward accounting is **backpropagation**. It uses the chain rule to **calculate one gradient for every trainable parameter**. It does not update a single parameter. Only after backprop has finished does **gradient descent apply those gradients** with $\theta \leftarrow \theta - \eta \nabla \theta$.

START WITH ONE FAMILIAR PATH

What if we turn just one weight?

one tiny nudge


inputs x_m
x1
0.50
x2
0.20
W_h1,x1 = 0.40
r_j → ReLU → h_j
h1
0.180
h2
0
W_sat,h1 = 0.80
logits z_i
sat
0.244
ran
0.036
slept
-0.122
purred
0.128
meowed
-0.038
probabilities q_i
Softmax q
q_sat=0.241



This is the exact network used by the equations below. Input index m selects x_m, hidden index j selects r_j and h_j, and word index i selects z_i, q_i, y_i. Every input connects to every hidden unit through W_{h_j,x_m}, with bias b_{h_j}; every hidden unit connects to every word logit through W_{i,h_j}, with bias b_i. The highlighted route is $x_1 \xrightarrow{W_{h1,x1}=0.40} r_1 \to h_1 \xrightarrow{W_{sat,h1}=0.80} z_{sat} \to q_{sat} \to L$, where $y_{sat} = 1$.

We have already pushed this example forward. The first hidden neuron is $h_1 = 0.180$. Its weight into the true word **sat** is $W_{sat,h1} = 0.800$, and the model ends at loss $L = 1.423$. Freeze every other number. Now raise only that one weight by 0.01 and watch the consequence travel to the right.

signal already computed

$h1 = 0.180$

↓

the knob we turn

$W_{sat,h1} = 0.800$

↓

raw sat score

$zsat = 0.244$

↓

sat probability

$qsat = 0.241$

↓

cross-entropy loss

$L = 1.423$

Before — the frozen forward pass

weight	sat logit
0.800	**0.2440**
sat probability	**loss**
0.2409	**1.4232**

After +0.01 — only this weight changed

weight	sat logit
0.810	**0.2458**
sat probability	**loss**
0.2413	**1.4219**

Auto-differentiation in libraries: PyTorch and TensorFlow record this dependency picture during the forward pass as a *computational graph*. Each operation remembers its local slope; reverse-mode automatic differentiation composes those slopes from L back to every parameter. That is how software performs backpropagation. It calculates the gradients; gradient descent applies them afterward to update the parameters.

NOW FOLLOW THE EFFECT BACKWARD

One gradient answers "how sensitive is the loss?"

The nudge experiment gives us a rate: the loss changed by about -0.001366 when the weight changed by $+0.01$. Their ratio is about -0.1366. Calculus asks what that ratio becomes for an infinitesimally small nudge and writes it as $\partial L/\partial W_{sat,h1}$.

The true score should rise; every competing score should fall.

This one example says **sat** really came next. Its target is 1, while every other target is 0. The size of each request is $q_i - y_i$: a badly wrong output asks for a larger correction; an already-good output asks for a smaller one.

sat
$0.241 - 1 = -0.759$
raise this score

ran
$0.196 - 0 = +0.196$
lower this score

slept
$0.167 - 0 = +0.167$
lower this score

purred
$0.215 - 0 = +0.215$
lower this score

meowed
$0.182 - 0 = +0.182$
lower this score

The signs are gradients, so gradient descent later moves in the opposite direction: subtracting the negative *sat* gradient raises its score; subtracting positive competitor gradients lowers theirs.

THE WHOLE ALGORITHM AT A GLANCE

Forward calculates the loss. Backward calculates every gradient.

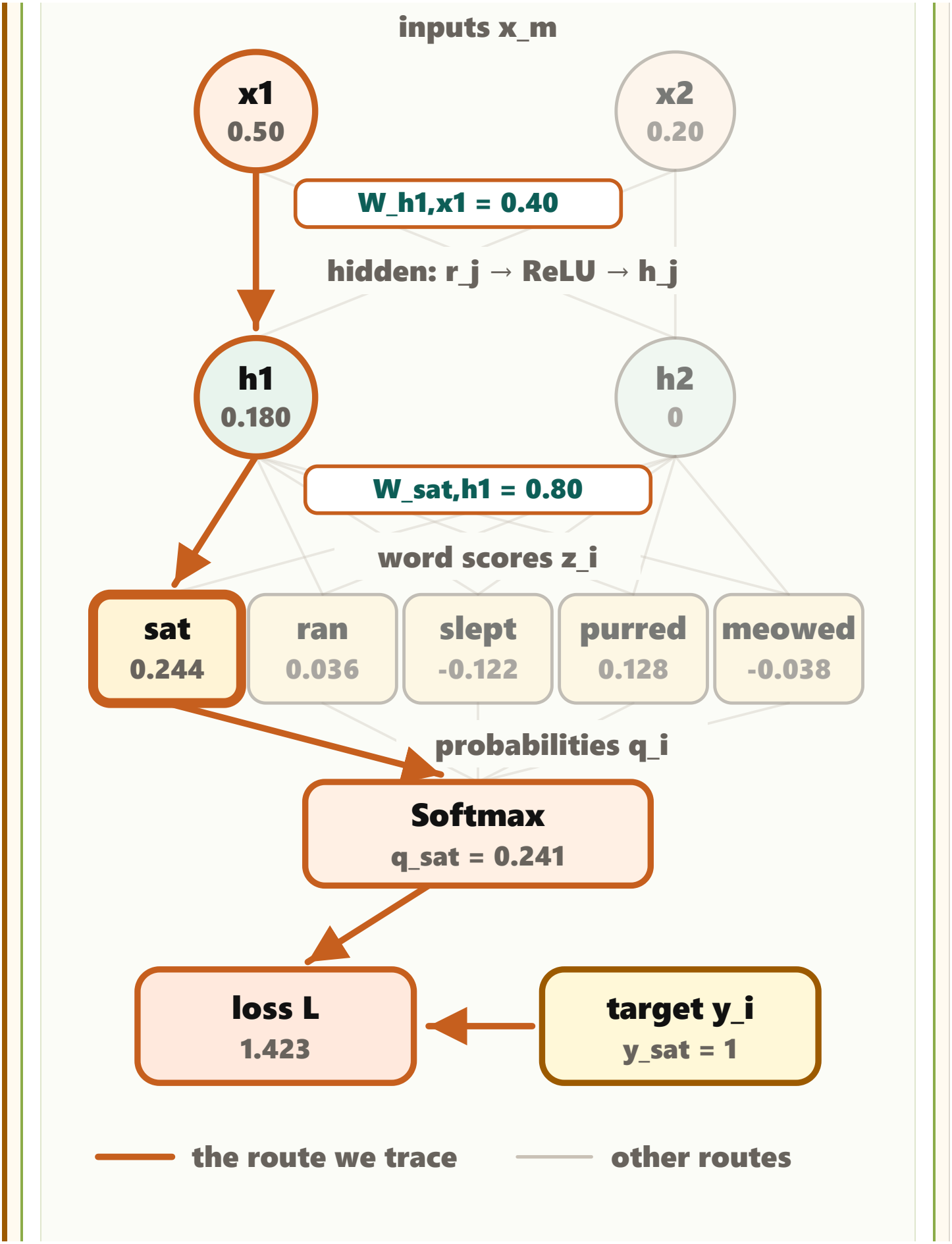
This is the complete scalar map for one training example in our two-input, two-hidden-unit, five-word network. Read the left column from inputs to loss, then read the right column from that loss back to every trainable number. The four steps below derive these same lines with the actual arithmetic. As everywhere in this worked example, temperature is fixed at $T = 1$.

Both directions visit the same four stages, so here they are once, side by side. Each column starts with a picture of that direction, so the numbers and variable names stay in front of you. Then read a row across: the forward equation on the left, and the derivative of *that exact equation* on the right. Forward runs 1 to 4 down the page; backward starts at the bottom row and climbs back up. Each output blame also drops off a parameter gradient as it passes: $\partial L/\partial W_{i,h_j} = \delta_i h_j = -0.137$ on the traced output weight.

The whole network, both ways

Every number in these two pictures, and in every row below, comes from the one example we have been following: the context `the cat`, with `sat` as the word that really came next.

Forward ↓ values fall from the inputs to the loss.


inputs x_m
x1
0.50
x2
0.20
W_h1,x1 = 0.40
hidden: r_j → ReLU → h_j
h1
0.180
h2
0
W_sat,h1 = 0.80
word scores z_i
sat
0.244
ran
0.036
slept
-0.122
purred
0.128
meowed
-0.038
probabilities q_i
Softmax
q_sat = 0.241
loss L
1.423
target y_i
y_sat = 1
the route we trace
other routes

Every input reaches every hidden unit, and every hidden unit reaches all five word scores. The bold route is the one we follow by hand.

Backward ↑ gradients climb the same wires, starting at the loss.

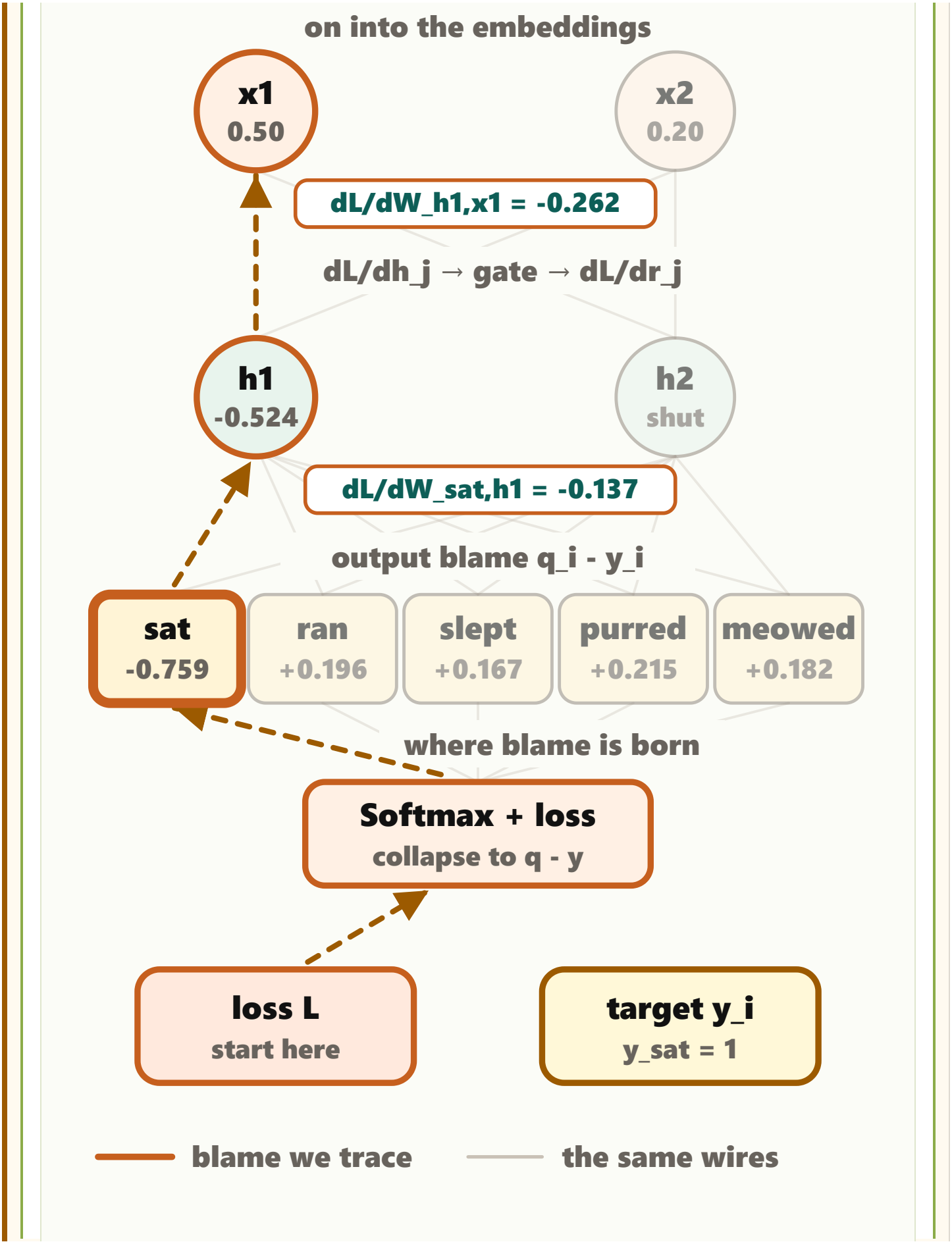
$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial L}{\partial \theta}$$

$\eta = 0.10$ $W_{h_1,x_1} : 0.400 \to 0.426$ $W_{\text{sat},h_1} : 0.800 \to 0.814$

$m = 1 \dots D$ $D = 2$ $j = 1 \dots H$ $H = 2$ $i, k = 1 \dots V$ $V = 5$

r h z q y δ

θ η


on into the embeddings
x1
0.50
x2
0.20
dL/dW_h1,x1 = -0.262
dL/dh_j → gate → dL/dr_j
h1
-0.524
h2
shut
dL/dW_sat,h1 = -0.137
output blame q_i - y_i
sat
-0.759
ran
+0.196
slept
+0.167
purred
+0.215
meowed
+0.182
where blame is born
Softmax + loss
collapse to q - y
loss L
start here
target y_i
y_sat = 1
blame we trace
the same wires

The wires never move. Only the direction changes, and each node now holds a gradient instead of a value.

Inputs and the hidden pre-activation

Forward 1. Combine every input at hidden unit j

$$r_j = \sum_{m=1}^{D} W_{h_j,x_m} x_m + b_{h_j}$$

Each hidden unit forms one weighted sum before its activation.

Backward 4. Finish at input-layer parameters and embeddings

$$\frac{\partial L}{\partial W_{h_j,x_m}} = \frac{\partial L}{\partial r_j} x_m$$

$$\frac{\partial L}{\partial b_{h_j}} = \frac{\partial L}{\partial r_j}$$

$$\frac{\partial L}{\partial x_m} = \sum_{j=1}^{H} \frac{\partial L}{\partial r_j} W_{h_j,x_m}$$

If x_m came from an embedding lookup, the last gradient continues into that embedding coordinate.

The hidden activation

Forward 2. Apply the ReLU gate

$$h_j = \mathrm{ReLU}(r_j) = \max(0, r_j)$$

Positive signals pass through; negative signals become zero.

Backward 3. Pass back through the same gate

$$\frac{\partial L}{\partial r_j} = \frac{\partial L}{\partial h_j} \mathrm{ReLU}'(r_j)$$

$$\mathrm{ReLU}'(r_j) = 1 \text{ if } r_j > 0;\ 0 \text{ if } r_j < 0$$

A closed gate blocks the gradient; an open gate passes it through. At exactly $r_j = 0$, we use the common software convention $\mathrm{ReLU}'(0) = 0$.

Hidden layer to word logits

Forward 3. Calculate every word logit i

$$z_i = \sum_{j=1}^{H} W_{i,h_j} h_j + b_i$$

The hidden signals combine into one raw score for each candidate word.

Backward 2. Calculate output parameters, then gather hidden blame

$$\frac{\partial L}{\partial W_{i,h_j}} = \delta_i h_j$$

$$\frac{\partial L}{\partial b_i} = \delta_i$$

1

Start with the loss, then find the blame on every output score.

Let us write down the complete forward calculation before taking a single derivative. We keep the same word order everywhere: **sat, ran, slept, purred, meowed**. The five raw output scores are the logits $\mathbf{z}$, and the one-hot target $\mathbf{y}$ puts a 1 beside the word that really came next:

$$z_{\text{sat}} = 0.244, \quad y_{\text{sat}} = 1$$

$$z_{\text{ran}} = 0.036, \quad y_{\text{ran}} = 0$$

$$z_{\text{slept}} = -0.122, \quad y_{\text{slept}} = 0$$

$$z_{\text{purred}} = 0.128, \quad y_{\text{purred}} = 0$$

$$z_{\text{meowed}} = -0.038, \quad y_{\text{meowed}} = 0$$

This example uses temperature $T = 1$, so Softmax exponentiates each logit and divides by one shared denominator. Call that denominator S:

$$S = \sum_j e^{z_j}$$

$$S = e^{0.244} + e^{0.036} + e^{-0.122} + e^{0.128} + e^{-0.038}$$

$$S \approx 1.276 + 1.037 + 0.885 + 1.137 + 0.963 = 5.297$$

$$q_i = \frac{e^{z_i}}{S}$$

$$q_{\text{sat}} = \frac{e^{0.244}}{5.297} = 0.241$$

$$q_{\text{ran}} = \frac{e^{0.036}}{5.297} = 0.196$$

$$q_{\text{slept}} = \frac{e^{-0.122}}{5.297} = 0.167$$

$$q_{\text{purred}} = \frac{e^{0.128}}{5.297} = 0.215$$

$$q_{\text{meowed}} = \frac{e^{-0.038}}{5.297} = 0.182$$

Cross-entropy now compares that probability distribution $\mathbf{q}$ with the target distribution $\mathbf{y}$. For one example its full loss equation is:

$$L = -\sum_i y_i \ln(q_i)$$

Only the **sat** term survives because its target is 1 and every competing target is 0:

$$-y_{\text{sat}} \ln(q_{\text{sat}}) = -(1) \ln(0.241) = 1.423$$

$$-y_{\text{ran}} \ln(q_{\text{ran}}) = -(0) \ln(0.196) = 0$$

$$-y_{\text{slept}} \ln(q_{\text{slept}}) = -(0) \ln(0.167) = 0$$

$$-y_{\text{purred}} \ln(q_{\text{purred}}) = -(0) \ln(0.215) = 0$$

$$-y_{\text{meowed}} \ln(q_{\text{meowed}}) = -(0) \ln(0.182) = 0$$

$$L = 1.423 + 0 + 0 + 0 + 0 = 1.423$$

Now we can ask how the loss changes when one logit changes. Substituting $q_{\text{sat}} = e^{z_{\text{sat}}}/S$ into the loss makes every dependency visible:

$$L = -\ln\left(\frac{e^{z_{\text{sat}}}}{S}\right)$$

$$L = -z_{\text{sat}} + \ln(S)$$

$$\frac{\partial S}{\partial z_i} = e^{z_i}$$

For the true-word logit, both terms in $L = -z_{\text{sat}} + \ln S$ depend on that logit. The derivative of the first term is -1; the derivative of the shared $\ln S$ term is its Softmax probability:

$$\frac{\partial L}{\partial z_{\text{sat}}} = -1 + \frac{1}{S}\frac{\partial S}{\partial z_{\text{sat}}}$$

$$\frac{\partial L}{\partial z_{\text{sat}}} = -1 + \frac{e^{z_{\text{sat}}}}{S} = -1 + q_{\text{sat}}$$

$$\delta_{\text{sat}} = \frac{\partial L}{\partial z_{\text{sat}}} = 0.241 - 1 = -0.759$$

For a competing logit such as **ran**, the $-z_{\text{sat}}$ term contributes 0 because it does not contain z_{ran}. Only the shared denominator changes:

$$\frac{\partial L}{\partial z_{\text{ran}}} = 0 + \frac{1}{S}\frac{\partial S}{\partial z_{\text{ran}}}$$

$$\frac{\partial L}{\partial z_{\text{ran}}} = \frac{e^{z_{\text{ran}}}}{S} = q_{\text{ran}} = 0.196$$

The same reasoning gives the compact rule $\delta_i = \partial L/\partial z_i = q_i - y_i$. Written out for all five outputs, the arriving gradient is:

$$\delta_{\text{sat}} = 0.241 - 1 = -0.759$$

$$\delta_{\text{ran}} = 0.196 - 0 = +0.196$$

$$\delta_{\text{slept}} = 0.167 - 0 = +0.167$$

$$\delta_{\text{purred}} = 0.215 - 0 = +0.215$$

$$\delta_{\text{meowed}} = 0.182 - 0 = +0.182$$

Negative blame on **sat** means raising its score would lower the loss. Positive blame on every competitor means raising that score would raise the loss. This is the complete source of the first chain-rule factor used in the next step.

2

Walk backward through this one weight.

We are differentiating the loss with respect to $W_{\text{sat},h1}$, the highlighted weight from h_1 to the **sat** logit. Its complete dependency path is:

$$W_{\text{sat},h1} \longrightarrow z_{\text{sat}} \longrightarrow \mathbf{q} \longrightarrow L$$

The first factor comes from the loss calculation we just derived:

$$L = -\sum_i y_i \ln(q_i) = -\ln(q_{\text{sat}})$$

$$\frac{\partial L}{\partial z_{\text{sat}}} = q_{\text{sat}} - 1$$

$$\frac{\partial L}{\partial z_{\text{sat}}} = 0.241 - 1 = -0.759$$

Next write the forward equation that this particular weight appears in. The **sat** logit receives both hidden signals and a bias:

$$z_{\text{sat}} = W_{\text{sat},h1} h_1 + W_{\text{sat},h2} h_2 + b_{\text{sat}}$$

In our frozen example, $h_1 = 0.180$, $h_2 = 0$, $W_{\text{sat},h1} = 0.800$, and $b_{\text{sat}} = 0.100$:

$$z_{\text{sat}} = (0.800)(0.180) + W_{\text{sat},h2}(0) + 0.100$$

$$z_{\text{sat}} = 0.144 + 0 + 0.100 = 0.244$$

When taking the partial derivative with respect to $W_{\text{sat},h1}$, we turn only that weight and hold every other quantity fixed. The terms containing $W_{\text{sat},h2}$ and the bias therefore contribute 0:

$$\frac{\partial (W_{\text{sat},h1} h_1)}{\partial W_{\text{sat},h1}} = h_1$$

$$\frac{\partial (W_{\text{sat},h2} h_2)}{\partial W_{\text{sat},h1}} = 0$$

$$\frac{\partial b_{\text{sat}}}{\partial W_{\text{sat},h1}} = 0$$

$$\frac{\partial z_{\text{sat}}}{\partial W_{\text{sat},h1}} = h_1 + 0 + 0 = 0.180$$

The chain rule now multiplies the loss sensitivity arriving at the logit by the logit's sensitivity to this weight:

$$\frac{\partial L}{\partial W_{\text{sat},h1}} = \frac{\partial L}{\partial z_{\text{sat}}} \frac{\partial z_{\text{sat}}}{\partial W_{\text{sat},h1}}$$

$$\frac{\partial L}{\partial W_{\text{sat},h1}} = (q_{\text{sat}} - 1) h_1$$

$$\frac{\partial L}{\partial W_{\text{sat},h1}} = (-0.759)(0.180) = -0.13662 \approx -0.137$$

That is the exact version of our experimental slope -0.1366. One path means multiply the local slopes along that path. Notice the $h_1 = 0.180$ factor: a weight carrying a larger incoming signal has more leverage, because the same tiny weight change produces a larger score change. Here $h_2 = 0$, so this example gives every h_2-to-word weight a gradient of 0.

> **Biases are the easy case.** A score bias is the same as a weight whose incoming signal is always 1. Since $z_{\text{sat}} = \cdots + b_{\text{sat}}$, the bias's local derivative is 1. Its complete chain is:

$$\frac{\partial z_{\text{sat}}}{\partial b_{\text{sat}}} = 1$$

$$\frac{\partial L}{\partial b_{\text{sat}}} = \frac{\partial L}{\partial z_{\text{sat}}} \frac{\partial z_{\text{sat}}}{\partial b_{\text{sat}}}$$

$$\frac{\partial L}{\partial b_{\text{sat}}} = (-0.759)(1) = -0.759$$

3

Move one layer earlier, where the path branches.

Hidden signal h_1 fed all five word scores. Each output sends its own request backward through its connecting weight. Those requests can agree or compete, so now the routes add. Start with the generic forward equation for each word i:

$$z_i = W_{i,h1}h_1 + W_{i,h2}h_2 + b_i$$

If we vary h_1 while holding h_2, the weights, and the biases fixed, the local derivative is the coefficient attached to h_1:

$$\frac{\partial z_i}{\partial h_1} = W_{i,h1}$$

$$\frac{\partial z_{\text{sat}}}{\partial h_1} = 0.80$$

$$\frac{\partial z_{\text{ran}}}{\partial h_1} = 0.20$$

$$\frac{\partial z_{\text{slept}}}{\partial h_1} = -0.40$$

$$\frac{\partial z_{\text{purred}}}{\partial h_1} = 0.60$$

$$\frac{\partial z_{\text{meowed}}}{\partial h_1} = -0.10$$

The multivariable chain rule says to follow every route from h_1 to the loss and add the results:

$$\frac{\partial L}{\partial h_1} = \sum_i \frac{\partial L}{\partial z_i}\frac{\partial z_i}{\partial h_1}$$

Now substitute the five arriving gradients from Step 1 and the five local slopes above:

sat route: $(-0.759)(0.80) = -0.6072$

ran route: $(+0.196)(0.20) = +0.0392$

slept route: $(+0.167)(-0.40) = -0.0668$

purred route: $(+0.215)(0.60) = +0.1290$

meowed route: $(+0.182)(-0.10) = -0.0182$

$-0.6072 + 0.0392 = -0.5680$

$-0.5680 - 0.0668 = -0.6348$

$-0.6348 + 0.1290 = -0.5058$

$-0.5058 - 0.0182 = -0.5240$

$$\frac{\partial L}{\partial h_1} \approx -0.5240$$

This is the second rule to keep: **multiply along one path; add where paths meet.** We cannot directly turn h_1; it is a value produced by earlier weights and a bias. The sum -0.524 is therefore a desired change that we carry one layer farther back to the parameters we can actually update.

4

Pass through the ReLU gate and reach the earlier weight.

We now continue from h_1 to the earlier weight $W_{h1,x1}$. First write the full hidden-neuron equation. The weighted input sum r_1 is the value *before* ReLU:

$$r_1 = W_{h1,x1}x_1 + W_{h1,x2}x_2 + b_{h1}$$

$$r_1 = (0.40)(0.50) + (-0.10)(0.20) + 0$$

$$r_1 = 0.200 - 0.020 = 0.180$$

$$h_1 = \mathrm{ReLU}(r_1) = \max(0, r_1) = 0.180$$

ReLU passes positive inputs through unchanged and replaces non-positive inputs with 0. Its local slope is therefore:

$$\frac{\partial h_1}{\partial r_1} = \mathrm{ReLU}'(r_1) = \begin{cases} 1, & r_1 > 0 \\ 0, & r_1 < 0 \end{cases}$$

Here $r_1 = 0.180 > 0$, so the gate is open and $\partial h_1/\partial r_1 = 1$. Next differentiate the weighted sum with respect to the one earlier weight we are following:

$$\frac{\partial (W_{h1,x1}x_1)}{\partial W_{h1,x1}} = x_1$$

$$\frac{\partial (W_{h1,x2}x_2)}{\partial W_{h1,x1}} = 0$$

$$\frac{\partial b_{h1}}{\partial W_{h1,x1}} = 0$$

$$\frac{\partial r_1}{\partial W_{h1,x1}} = x_1 + 0 + 0 = 0.50$$

We now have all three factors, each traced to the equation that created it:

$$\frac{\partial L}{\partial W_{h1,x1}} = \frac{\partial L}{\partial h_1}\frac{\partial h_1}{\partial r_1}\frac{\partial r_1}{\partial W_{h1,x1}}$$

$$\frac{\partial L}{\partial W_{h1,x1}} \approx (-0.524)(1)(0.50)$$

$$\frac{\partial L}{\partial W_{h1,x1}} \approx -0.262$$

Backpropagation has now **calculated** this weight's gradient. Only afterward does gradient descent **apply** it. With learning rate $\eta = 0.10$, the general update rule and this specific update are:

$$W_{\text{new}} = W_{\text{old}} - \eta\frac{\partial L}{\partial W}$$

$$W_{h1,x1}^{\text{new}} = 0.400 - (0.10)(-0.262)$$

$$W_{h1,x1}^{\text{new}} = 0.400 + 0.0262 = 0.4262 \approx 0.426$$

The gradient is negative, so subtracting it increases the weight. That is not a separate sign trick: it is exactly what the downhill update rule says to do when increasing this weight is predicted to lower the loss.

ONE EXAMPLE CASTS ONE VOTE

This backward pass does not speak for the whole corpus.

Every number above answers: "How should the network change for this one example, where **sat** was the true next word?" Another example may ask the same weight to move differently. Full-batch gradient descent averages all those per-example gradients before one update. Mini-batches average a smaller random group; the browser trainer later uses one-example-at-a-time SGD, so it applies each noisy vote immediately.

Let $s_i = z_i/T$ be the temperature-scaled score and let c be the true-word index. With $q_i = e^{s_i}/\sum_j e^{s_j}$, the one-example cross-entropy loss can be rewritten as:

$$L = -\ln q_c = -s_c + \ln \sum_j e^{s_j}$$

The first term contributes $-y_i$; the shared log-sum contributes q_i:

$$\frac{\partial L}{\partial s_i} = -y_i + \frac{e^{s_i}}{\sum_j e^{s_j}} = q_i - y_i$$

Finally, $\partial s_i/\partial z_i = 1/T$, so the general result is:

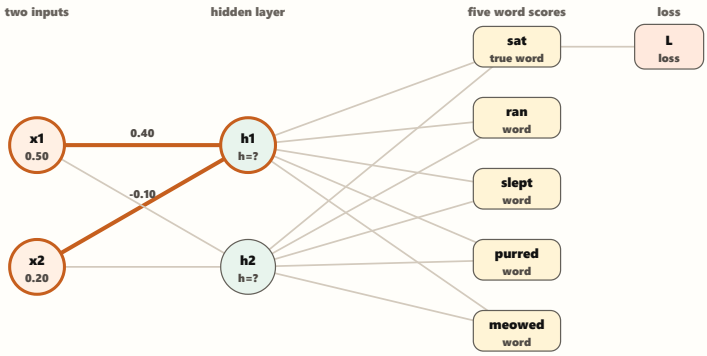
$$\frac{\partial L}{\partial z_i} = \frac{q_i - y_i}{T}$$

Here $T = 1$, so the factor is 1. The shared Softmax denominator is why changing one score affects every probability; the compact $q - y$ vector already accounts for that coupling.

SOLVED WORKSHEET TRANSCRIPT

Do one complete training step by hand - all 13 solutions

The live worksheet lets you attempt each number, ask for progressive hints, and receive immediate feedback: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html



`x1=0.50, x2=0.20`

INPUTS

`"sat"`

TARGET

`compute h`

HIDDEN STATE

`0.10`

LEARNING RATE

STEP 1 OF 13

Step 1: find the raw value entering hidden neuron h1.

Use the two incoming edges: x1 x 0.40 plus x2 x -0.10. With x1 = 0.50 and x2 = 0.20, what enters h1 before ReLU?

Solution revealed. 0.50 x 0.40 + 0.20 x -0.10 = 0.180

STEP 2 OF 13

Step 2: pass h1 through the ReLU gate.

ReLU keeps positive numbers and turns negative numbers into 0. The raw h1 value is 0.180. What is h1 after ReLU?

Solution revealed. ReLU(0.180) = 0.180

STEP 3 OF 13

Step 3: find the raw value entering hidden neuron h2.

Now use the second hidden neuron: x1 x -0.30 plus x2 x 0.70. What enters h2 before ReLU?

Solution revealed. 0.50 x -0.30 + 0.20 x 0.70 = -0.010

STEP 4 OF 13

Step 4: pass h2 through the ReLU gate.

The raw h2 value is -0.010. ReLU shuts negative values off. What is h2 after ReLU?

Solution revealed. ReLU(-0.010) = 0

STEP 5 OF 13

Step 5: calculate the score for the word "sat".

Use z_sat = h1 x 0.80 + h2 x -0.20 + bias 0.10. With h1 = 0.180 and h2 = 0, what is the logit for "sat"?

Solution revealed. 0.180 x 0.80 + 0 x -0.20 + 0.10 = 0.244

STEP 6 OF 13

Step 6: build the softmax denominator.

The five logits are 0.244, 0.036, -0.122, 0.128, and -0.038. Add e raised to each logit. What is the denominator?

Solution revealed. e^0.244 + e^0.036 + e^-0.122 + e^0.128 + e^-0.038 = 5.297

STEP 7 OF 13

Step 7: turn the "sat" score into a probability.

Use q_sat = e^0.244 / 5.297. What probability does the model give the correct word?

Solution revealed. q_sat = 1.277 / 5.297 = 0.241

STEP 8 OF 13

Step 8: calculate the loss.

For the true word, cross-entropy is -ln(q_true). Use q_sat = 0.241. What is the loss in nats?

Solution revealed. -ln(0.241) = 1.423

STEP 9 OF 13

Step 9: find the output blame for "sat".

Softmax plus cross-entropy gives $\delta_i = (q_i - y_i) / T$ on each original logit. This worksheet uses $T = 1$, so the factor is 1. For the true word "sat", the target is 1. What blame flows backward from its score?

First name the pieces. z_{word} is a raw score, also called a logit. q_{word} is the probability after softmax. y_{word} is the target label: 1 for the correct word and 0 for every other word. δ_{word} is the blame sent backward into that word's raw score.

$$q_i = \text{softmax}(z)_i = e^{z_i} / \Sigma_k e^{z_k}$$

$$L = -\Sigma_j y_j \log(q_j)$$

We want the slope of the loss with respect to one raw score z_i. Because softmax shares one denominator, changing z_i changes every probability q_j, not just q_i.

$$\delta_i = \partial L / \partial z_i = \Sigma_j (\partial L / \partial q_j)(\partial q_j / \partial z_i)$$

Now get the cross-entropy derivative with respect to one probability. Focus on one term, $-y_j \log(q_j)$. The target y_j is just a constant multiplier here.

$$\partial / \partial q_j [-y_j \log(q_j)] = -y_j \times \partial \log(q_j) / \partial q_j$$

Why is the log slope $1 / q_j$? Let $u = \log(q_j)$. Natural log means the inverse of exponentiation, so $e^u = q_j$.

$$e^u = q_j$$

$$\partial / \partial q_j [e^u] = \partial / \partial q_j [q_j]$$

$$e^u (\partial u / \partial q_j) = 1$$

$$\partial u / \partial q_j = 1 / e^u = 1 / q_j$$

Substitute that log slope back into the cross-entropy term.

$$\partial L / \partial q_j = -y_j / q_j$$

The softmax part has two cases: the probability for the same word rises when its

STEP 10 OF 13

Step 10: get the gradient for the h1-to-sat weight.

This asks: if we nudge the weight from **h1** into the **sat** score, how much does the loss change? That sensitivity is written $\partial L / \partial W_{sat,h1}$. Use sat blame -0.759 and h1 = 0.180.

First name the pieces. $W_{sat,h1}$ is the weight from hidden activation h1 into the raw sat score. h_1 is the activation that flowed through that weight. δ_{sat} is shorthand for $\partial L / \partial z_{sat}$, the blame already sitting at the sat score.

The weight affects the loss only by first changing the raw sat score z_{sat}.

$$z_{sat} = W_{sat,h1} h_1 + W_{sat,h2} h_2 + b_{sat}$$

We want the slope of the loss with respect to this one weight. Follow the path backward: $L \rightarrow z_{sat} \rightarrow W_{sat,h1}$.

$$\partial L / \partial W_{sat,h1} = (\partial L / \partial z_{sat}) \times (\partial z_{sat} / \partial W_{sat,h1})$$

The first slope is the sat blame:

$$\partial L / \partial z_{sat} = \delta_{sat}$$

For the second slope, treat every other term in z_{sat} as constant. The only term containing $W_{sat,h1}$ is $W_{sat,h1} h_1$.

$$\partial z_{sat} / \partial W_{sat,h1} = h_1$$

$$\partial L / \partial W_{sat,h1} = \delta_{sat} \times h_1$$

Solution revealed. $\partial L / \partial W_{sat,h1} = -0.759 \times 0.180 = -0.137$

own score rises, while the other probabilities fall because the denominator got bigger.

$$\partial q_j / \partial z_i = q_j(1 - q_i) \text{ if } i = j$$

$$\partial q_j / \partial z_i = -q_j q_i \text{ if } i \neq j$$

A compact way to write both cases is:

$$\partial q_j / \partial z_i = q_j(1_{i=j} - q_i)$$

Now substitute both derivative pieces into the chain-rule sum.

$$\partial L / \partial z_i = \Sigma_j \, (-y_j / q_j) \, q_j(1_{i=j} - q_i)$$

$$= \Sigma_j \, -y_j(1_{i=j} - q_i)$$

$$= -y_i + q_i \Sigma_j y_j$$

For one correct next word, the target vector has exactly one 1, so $\Sigma_j y_j = 1$.

$$\partial L / \partial z_i = q_i - y_i$$

That is the blame on the scaled score. If temperature first made $s_i = z_i / T$, one final chain-rule factor gives $\partial L / \partial z_i = (q_i - y_i) / T$. Here $T = 1$, so output blame is predicted probability minus target.

Solution revealed. $\delta_{sat} = 0.241 - 1 = -0.759$

STEP 11 OF 13

Step 11: gather all the blame arriving at h1.

Hidden neuron **h1** helped all five word scores, so its blame is the sum of every output blame times the matching outgoing weight. What is $\partial L / \partial h_1$, the blame on h1 before the ReLU gate?

First name the pieces. h_1 is the hidden activation. z_{word} is a raw score for one output word. δ_{word} means $\partial L / \partial z_{word}$, the blame at that word score. $W_{word,h1}$ is the weight from h1 into that word score.

Hidden neuron h1 fans out into every word score. Changing h1 changes sat, ran, slept, purred, and meowed at the same time.

When one variable affects the loss through multiple paths, the chain rule adds the path contributions from all paths.

$$\partial L / \partial h_1 = \Sigma_{word} (\partial L / \partial z_{word}) \times (\partial z_{word} / \partial h_1)$$

The first piece on each path is that word's output blame:

$$\partial L / \partial z_{word} = \delta_{word}$$

For the second piece, write one output score. The term containing h1 is the outgoing weight times h1.

$$z_{word} = W_{word,h1} h_1 + W_{word,h2} h_2 + b_{word}$$

$$\partial z_{word} / \partial h_1 = W_{word,h1}$$

$$\partial L / \partial h_1 = \Sigma_{word} \delta_{word} \times W_{word,h1}$$

Solution revealed. $\partial L / \partial h_1 = (-0.759 \times 0.80) + (0.196 \times 0.20) + (0.167 \times -0.40) + (0.215 \times 0.60) + (0.182 \times -0.10) = -0.524$

STEP 12 OF 13

Step 12: turn h1's blame into the x1-to-h1 gradient.

Now move one edge farther backward. We want the gradient for the weight from **x1** into **h1**, written $\partial L / \partial W_{h1,x1}$. Because h1's ReLU gate was open, its slope is 1.

First name the pieces. r_1 is the raw value going into hidden neuron h1. h_1 is the value after ReLU. $W_{h1,x1}$ is the input weight from x1 into h1. x_1 is the input signal.

The input weight does not touch h1 directly. It first changes the raw hidden value r_1; then ReLU turns r_1 into h_1.

$$r_1 = W_{h1,x1} x_1 + W_{h1,x2} x_2 + b_{h1}$$

$$h_1 = ReLU(r_1)$$

The chain rule follows the path backward: $L \rightarrow h_1 \rightarrow r_1 \rightarrow W_{h1,x1}$.

$$\partial L / \partial W_{h1,x1} = (\partial L / \partial h_1) \times (\partial h_1 / \partial r_1) \times (\partial r_1 / \partial W_{h1,x1})$$

The middle slope is the ReLU gate. ReLU's derivative is 1 when the raw value is positive and 0 when the raw value is negative or zero.

$$\partial h_1 / \partial r_1 = ReLU'(r_1)$$

The last slope is x_1, because the term containing this weight is $W_{h1,x1} x_1$.

$$\partial r_1 / \partial W_{h1,x1} = x_1$$

$$\partial L / \partial W_{h1,x1} = (\partial L / \partial h_1) \times ReLU'(r_1) \times x_1$$

Solution revealed. $\partial L / \partial W_{h1,x1} = -0.524 \times 1 \times 0.50 = -0.262$

STEP 13 OF 13

Step 13: update the x1-to-h1 weight.

Gradient descent now uses that gradient to move the weight downhill. The old weight is 0.400, the learning rate is 0.10, and the gradient is -0.262. What is the new **x1** → **h1** weight?

First name the pieces. **W** is the weight we are updating. **g** is its gradient, meaning **g = ∂L / ∂W**. **η** is the learning rate, the size of the step.

The gradient points uphill: it tells us which direction makes the loss increase fastest.

For a tiny change **ΔW**, calculus gives this local approximation:

$$L(W + \Delta W) \approx L(W) + g\Delta W$$

To move downhill, choose a change that makes **gΔW** negative. Gradient descent chooses the opposite direction of the gradient.

$$\Delta W = -\eta g$$

Substitute that into the local approximation:

$$L(W + \Delta W) \approx L(W) - \eta g^2$$

Since g^2 is never negative, this small step is aimed downhill.

$$W_{new} = W_{old} + \Delta W = W_{old} - \eta g$$

$$W_{new} = W_{old} - \eta(\partial L / \partial W)$$

Solution revealed. W_{new} = 0.400 - 0.10 × (-0.262) = 0.426

Nothing new is happening here. We are doing every path at once.

The worksheet used input width $D = 2$, hidden width $H = 2$, and vocabulary size $V = 5$. It calculated one output-weight gradient and one input-weight gradient as scalars. A real implementation needs all ten entries of W_2 and all four entries of W_1, so linear algebra packs those repeated scalar operations into grids.

And ten is nothing. A real language model does not have ten weights, it has billions. Walking them one at a time in a `for` loop is not slow, it is hopeless - you would still be waiting years later. Matrices are how we say "do all of that at once", and

the hardware in a GPU is built for exactly this shape of work. So the goal here is not new math. It is the same arithmetic you just did by hand, written so a machine can do a million copies of it in one breath.

THE WORKSHEET'S NUMBER ARRANGEMENTS

Every result must match the parameter grid it will update.

`x: D×1 = 2×1`

two input signals

`W1: H×D = 2×2`

input-to-hidden weights

`h: H×1 = 2×1`

two hidden signals

`W2: V×H = 5×2`

hidden-to-word weights

`z,q,δ: V×1 = 5×1`

one value per word

Transpose $(\top)$

Flip rows into columns. $W_2^\top \delta$ performs the five-path sum for both hidden neurons.

Element-wise multiply $(\odot)$

Pair matching positions. Each hidden blame meets its own ReLU slope.

Outer product

Multiply every blame by every incoming signal. The result has one gradient for every weight.

Forward, the shapes fit. Turn around and they clash.

This is the moment the transpose earns its keep, so it is worth watching closely. Going forward, nothing is awkward. Lay the 5×2 output grid against the 2×1 hidden column, add the bias, and out comes exactly one score per word - the same five logits the worksheet produced:

$$\underbrace{\begin{bmatrix} 0.8 & -0.2 \\ 0.2 & 0.5 \\ -0.4 & 0.1 \\ 0.6 & -0.3 \\ -0.1 & 0.4 \end{bmatrix}}_{W_2\ (5\times2)} \underbrace{\begin{bmatrix} 0.180 \\ 0 \end{bmatrix}}_{h\ (2\times1)} + \underbrace{\begin{bmatrix} 0.10 \\ 0 \\ -0.05 \\ 0.02 \\ -0.02 \end{bmatrix}}_{b_2\ (5\times1)} = \underbrace{\begin{bmatrix} 0.244 \\ 0.036 \\ -0.122 \\ 0.128 \\ -0.038 \end{bmatrix}}_{z\ (5\times1)}$$

Now turn around. We are holding five blames, one per word, stacked in a 5×1 column, and they need to travel back into two hidden neurons. Try it with the grid as it stands and the multiplication is simply not legal - a 5×2 grid has nothing to pair with a 5×1 column:

$$\underbrace{(5 \times 2)}_{W_2} \times \underbrace{(5 \times 1)}_{\delta} \Rightarrow \text{undefined} \qquad \underbrace{(2 \times 5)}_{W_2^\top} \times \underbrace{(5 \times 1)}_{\delta} = \underbrace{(2 \times 1)}_{\text{blame at } h}$$

Nothing is broken. The wires are the same wires; we are just reading them in the other direction, so we lay the grid on its side first. That sideways flip is the whole trick, and the next three steps are simply that idea applied carefully.

1. Start with the five output blames

In word order **sat, ran, slept, purred, meowed**, the worksheet produced $\delta = q - y$:

$$\delta = \begin{bmatrix} -0.759 \\ 0.196 \\ 0.167 \\ 0.215 \\ 0.182 \end{bmatrix} \qquad (5 \times 1)$$

The negative *sat* entry says its score should rise. Every other entry is positive, so those competing scores should fall.

2. Transpose gathers every route back to the hidden layer

The five output rows of the worksheet's weight grid are:

$$W_2 = \begin{bmatrix} 0.8 & -0.2 \\ 0.2 & 0.5 \\ -0.4 & 0.1 \\ 0.6 & -0.3 \\ -0.1 & 0.4 \end{bmatrix} \qquad (5 \times 2)$$

Transposing lays each hidden neuron's five outgoing weights across one row. Each row-dot-column calculation is exactly the path sum we did for h_1:

$$W_2^\top \delta = \underbrace{\begin{bmatrix} 0.8 & 0.2 & -0.4 & 0.6 & -0.1 \\ -0.2 & 0.5 & 0.1 & -0.3 & 0.4 \end{bmatrix}}_{2\times5} \underbrace{\begin{bmatrix} -0.759 \\ 0.196 \\ 0.167 \\ 0.215 \\ 0.182 \end{bmatrix}}_{5\times1} = \underbrace{\begin{bmatrix} -0.524 \\ 0.275 \end{bmatrix}}_{2\times1}$$

That first -0.524 is the worksheet's scalar sum. The same multiplication calculated h_2's incoming blame at the same time.

3. Element-wise multiplication applies one activation gate per neuron

The raw hidden values were $[0.180, -0.010]^\top$. ReLU was open for h_1 and closed for h_2, so its two local slopes are $[1, 0]^\top$:

$$\delta^{(1)} = (W_2^\top \delta) \odot f'(r) = \begin{bmatrix} -0.524 \\ 0.275 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.524 \\ 0 \end{bmatrix}$$

The $\odot$ symbol does not mix positions. It pairs the first blame with the first gate and the second blame with the second gate. The closed gate blocks h_2's blame from reaching its incoming weights.

4. Outer products build one gradient for every edge

At the output layer, stand the five blames vertically and lay the two hidden signals horizontally. Every blame multiplies every signal:

$$\nabla W_2 = \delta h^\top = \begin{bmatrix} -0.137 & 0 \\ 0.035 & 0 \\ 0.030 & 0 \\ 0.039 & 0 \\ 0.033 & 0 \end{bmatrix} \qquad (5 \times 2)$$

The top-left -0.137 is the exact output-weight gradient we derived. The second column is zero because $h_2 = 0$, so no signal flowed through those five edges during this example.

At the input layer, do the same with two hidden deltas and two inputs:

$$\nabla W_1 = \delta^{(1)} x^\top = \begin{bmatrix} -0.262 & -0.105 \\ 0 & 0 \end{bmatrix} \qquad (2 \times 2)$$

The top-left -0.262 is the earlier gradient from our scalar path. Each gradient grid has exactly the same shape as the weight grid it will update.

Let one linear layer be $z = Wx$, with W shaped $M \times N$, x shaped $N \times 1$, and incoming blame $\delta = \partial L/\partial z$ shaped $M \times 1$.

Why $\nabla W = \delta x^\top$

A specific weight W_{ij} reaches only output z_i:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial z_i}\frac{\partial z_i}{\partial W_{ij}} = \delta_i x_j$$

An outer product places $\delta_i x_j$ at every row i, column j, producing the full $M \times N$ gradient grid.

Why the earlier blame is $W^\top \delta$

One input x_j fans out to all M outputs, so every path must contribute:

$$\frac{\partial L}{\partial x_j} = \sum_{i=1}^{M} \frac{\partial L}{\partial z_i}\frac{\partial z_i}{\partial x_j} = \sum_{i=1}^{M} \delta_i W_{ij}$$

Column j of W contains those connecting weights. Transpose puts that column into row j, so ordinary matrix multiplication performs every path sum for every input at once: $\partial L/\partial x = W^\top \delta$.

The two scalable rules

Build weight gradients - the weight-update rule: incoming blame column times incoming signal row.

$$\nabla W = \delta x^\top$$

Pass blame backward - the blame-passing rule: transposed weights gather the path sums, then the activation slopes gate them.

$$\delta_{earlier} = (W^{\top}\delta) \odot f'(r)$$

Every layer in every deep network you will ever meet is doing these two lines, over and over, from the loss back to the first weight.

Why deep networks need help keeping the signal alive

Every layer multiplies by weights and activation slopes. Across many layers, repeatedly multiplying by small slopes can shrink the backward signal until early layers barely learn. This is the vanishing gradient problem. Chapter 5 will show how skip connections and normalization help information and gradients travel through much deeper networks.

NOW WIDEN THE SAME COMPUTATION

The live panel uses more numbers, not a different algorithm.

The worksheet used $D = 2, H = 2, V = 5$ for hand arithmetic. This panel returns to the chapter's two 2D embedding rows joined into $D = 4$, with $H = 3$ hidden neurons and three visible candidate words. The graph, transpose, element-wise gate, outer products, and $T = 1$ output blame are the same. It also lets you switch the activation f, where the worksheet stayed on ReLU because its slope is a tidy 1 or 0; the trace below still calls the pre-activation r and the word scores z, exactly as above.

Deliberate before-state: the untrained gradients are populated on load because they are the evidence you compare against after pressing *Take a gradient step*.

Try the controls and motion in the live chapter: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

Watch the error flow backward

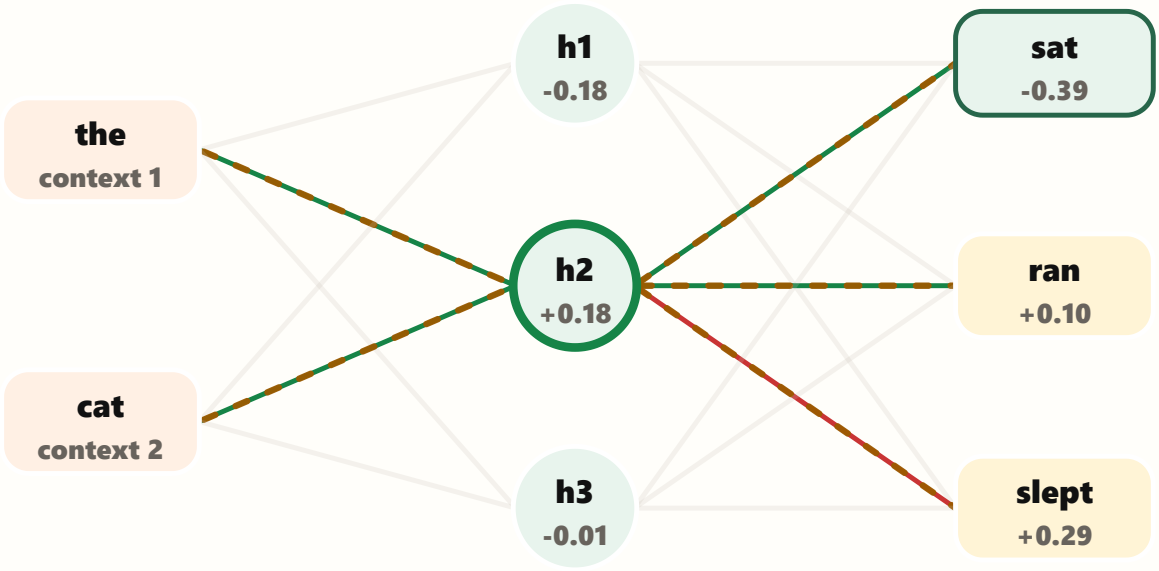
before training

Context 1

Context 2

True next word

Activation



The orange dashes flow **right to left** - that is the error travelling backward. Each output word shows its **blame** (`probability − target` because this panel uses `T = 1`); each hidden neuron shows the gradient that will nudge its weights. **Click a hidden neuron** to trace its chain rule on the right.

Learning rate `0.30`

0.492
CROSS-ENTROPY LOSS (NATS)
press Step

61%
P("SAT")
goal: 100%

0
GRADIENT STEPS

the loss curve appears as you take steps

The chain rule, step by step

blame → update

Every number is the real gradient for the network on the left, for the example you chose. Follow how the output blame is handed back, shrunk by the activation gate, and finally turned into an actual weight change.

1 - The starting error: probability minus target

word	q (pred)	y (target)	blame q − y
sat	0.612	1	−0.388
ran	0.101	0	+0.101
slept	0.287	0	+0.287

2 - Hand the blame back to hidden neuron h2

```
dL/dh = sum(output blame x W2)
```

```
dL/dh = (−0.39)x(−0.60) + 0.10x0.80 + 0.29x(−0.30)
```

```
dL/dh = +0.228
```

3 - Shrink it through the tanh gate

```
tanh'(r) at r = −0.48 = 0.801
```

```
dL/dr = dL/dh x tanh'(r) = +0.228 x 0.801 = +0.182
```

tanh's slope is at most 1 and falls toward 0 as the neuron saturates, easing the blame as it passes.

4 - Turn that into weight gradients

```
grad W2[h2 -> sat] = h x blame = (−0.45)x(−0.39) = +0.173
```

```
grad W1[the.x -> h2] = input x dL/dr = 0.20x0.18 = +0.036
```

5 - The update: each weight steps against its gradient

weight	old	− lr x grad	new
W2 h2->sat	−0.600	−0.052	−0.652
W1 the.x->h2	−0.600	−0.011	−0.611

> Right now the model gives "sat" only 61%. Press Take a gradient step to push every weight a little against its gradient, then watch this same number climb and the loss fall.

> Press *Take a gradient step* and every weight moves a little against its gradient. Watch the loss fall and the true word's probability climb - repeat it and this one example is slowly memorised, which is exactly

what training does, only across millions of examples at once.

Quick check: Why do we pass the error *backward* through the network to calculate gradients, rather than just adjusting weights and running it forward to check the result?

It's the only mathematically valid way to use calculus.

Efficiency: One backward sweep finds the exact gradient for every weight at once.

It only updates the final output layer.

Answer: Efficiency: One backward sweep finds the exact gradient for every weight at once.. By calculating the error at the end and flowing it backward, we can find the exact gradient for every single weight in the network in just one sweep. Going forward would require tweaking every weight individually to see what happens.

7. THE FINAL BUILD

Train a real next-word model on an entire book.

This is the payoff: everything above becomes one small neural language model that predicts the next token. It does not train on a hand-written demonstration or stop when an arbitrary timer expires. Choose one of the public-domain books used in the earlier chapters, and each pass processes **every token in that selected text**.

Every training example runs the complete pipeline in compute order: **embedding lookup** → **forward pass** → **Softmax** → **cross-entropy** → **backpropagation** → **gradient descent**. One **pass**, also called an epoch, means visiting every context-to-next-token example once. The browser shuffles those examples and updates after one example at a time, which is stochastic gradient descent. Larger systems usually group examples into mini-batches so accelerators can work in parallel.

> *An honest limit:* the model reads the whole book, but full Softmax over thousands of rare spellings would make this small browser exercise needlessly slow. It keeps the 511 most frequent tokens plus one visible `<rare>` token. Every original token is still processed; rarer spellings share that one vocabulary row. The live coverage readout below shows exactly how much of the book remains individually named.

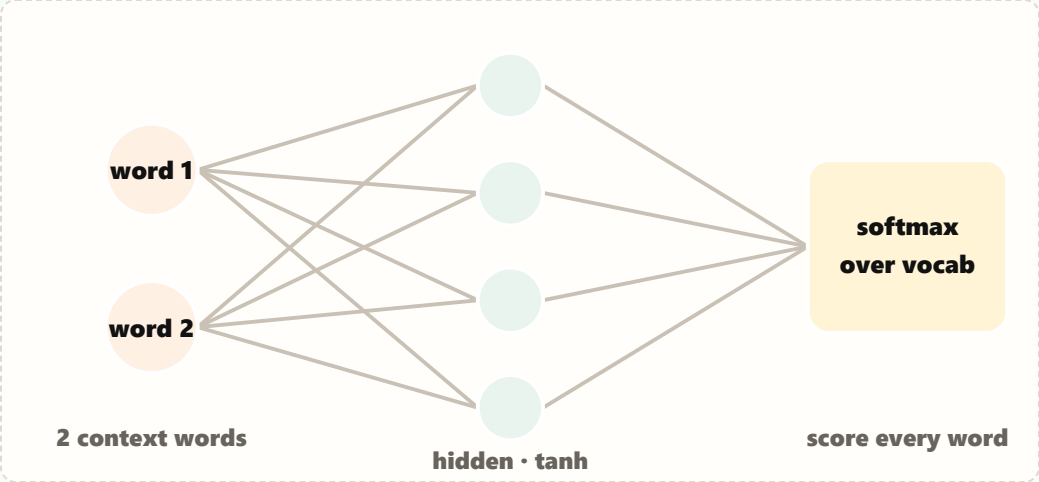
> *Training is not evaluation.* Falling training loss proves that the model is fitting this book, not that it will generalize to unseen writing. Chapter 1's **held-out evaluation** is still the honest test.

WHAT YOU ARE TRAINING

A tiny next-word network - the exact forward pass above, with learning switched on.

No mystery box here. The two context words are each looked up as a learned **vector** (an embedding). The two vectors are joined, pushed through a **hidden layer** of `tanh` neurons, and turned into one score for every word in the vocabulary. Softmax makes those scores a probability distribution: "given these two words, here is how likely each possible next word is."

Training nudges every number along that path - the embeddings, both weight layers, and the biases - using the exact backprop you just traced. This browser model uses 12 numbers per embedding, a two-token context, 24 hidden units, and a 512-token output vocabulary. The live readout below counts every parameter and shows how the selected book becomes training examples.



Two context words → embeddings → one hidden layer (tanh) → softmax over every word in the vocabulary. The *parameters* readout below is exactly these weights.

Choose the book this model will actually read

34,200 tokens

Public-domain corpus

The preview is short; training uses the complete selected text. Open Lewis Carroll.

```
alice’s adventures in wonderland by lewis carroll the millennium fulcrum edition 3. 0 contents chapter i. down the rabbit - hole chapter ii. the pool of tears chapter iii
```

```
. a caucus - race and a long tale chapter iv. the rabbit sends in a little bill chapter v. advice from a caterpillar chapter vi. pig and pepper
```

```
chapter vii. a mad tea - party chapter viii. the queen’s croquet - ground chapter ix. the mock turtle’s story chapter x. the lobster quadrille chapter xi.
```

34,200 **book tokens** all processed each pass	**34,198** **training examples** two-token context → target
2,653 **raw token types** before the browser cap	**512** **model vocabulary** 511 frequent + <rare>
87% **named-token coverage** the rest map to <rare>	**19,544** **parameters** every learned number

Fixed architecture: two-token context, 12 numbers per embedding, 24 tanh hidden units, full Softmax over all 512 model tokens, and learning rate 0.015. These are browser-speed choices, not production recommendations.

Try the controls and motion in the live chapter: https://talktovishal.github.io/projects/llm-soup-to-nuts/chapter-4.html

Training controls

Choose complete passes, then train. Every pass shuffles and visits all 34,198. Progress reports the exact position in the selected book-derived dataset.

Complete passes

Generation temperature `0.80`

That temperature is the same dial you turned in Softmax and cross-entropy: it divides the logits before Softmax, so a low value makes the model play it safe and a high value makes it take chances. It only affects the sentences we generate below, never the training itself.

> **Reset contract:** *Reset model* restores the original random weights and clears passes, metrics, loss history, predictions, and generated text. Changing the book also creates a fresh model for that corpus.

This lab uses the simplest optimizer: subtract the current gradient times one fixed learning rate.

- **Momentum** keeps a running direction so noisy updates do not zigzag as much.
- **RMSprop** scales each parameter using its recent squared gradients, giving different knobs different effective step sizes.
- **Adam** combines momentum-like direction with adaptive scaling.
- **AdamW** keeps Adam's adaptive update but separates weight decay, making regularization easier to control.

Those methods change how gradients become steps. They do not replace the forward pass, loss, or backpropagation that calculates the gradients.

Ready to train

Choose complete passes and press Train. Each pass will visit all 34,198 examples.

Predict the next word

said alice →

Context 1

Context 2

The selected context updates during training so you can watch the model's probabilities sharpen.

.



26%

,



16%

<rare>



12%

;



8%

:



4%

to



3%

Training signal

2 passes

3.792
SAMPLED TRAIN LOSS
1,024 evenly spaced examples

44.34
SAMPLED PERPLEXITY
effective token choices

2
COMPLETE PASSES
every book token per pass

34,200
BOOK TOKENS
all used for training

512
MODEL VOCABULARY
including <rare>

2 tokens
CONTEXT
fixed window

random



6.274

pass 1



3.985

pass 2



3.792

Generated continuation

sampled

said alice at the duchess. a ” * * * * * * * * * * * * the large.

sampled at temperature 0.80 · <rare> is excluded because it cannot decode to one specific spelling · click again for another

Take a moment to appreciate what just happened. With probability, vectors, dot products, Softmax, cross-entropy, backpropagation, and SGD, a pile of numbers learned directly from a real book. This model is still deliberately tiny: rare spellings share one token and every prediction must squeeze the past into exactly two neighboring tokens. Chapter 5 introduces **attention**, which gathers useful context dynamically from farther back while keeping the machinery you built here. The training loop stays; the way context enters the guess changes.

8. GOING DEEPER

Further reading.

If the live examples now feel comfortable, I would go deeper in this order: first the paper behind the little model you just trained, then the classic sources on backpropagation, softmax, and perplexity. You do not need to read them to keep going, but they are the real trailheads.

1. **Ronald T. Kneusel, Math for Deep Learning**
   A practical foundation for this chapter: Chapters 5–6 cover linear algebra and matrix operations, Chapters 7–8 cover differential and matrix calculus, Chapter 10 develops backpropagation, and Chapter 11 covers gradient descent and adaptive optimizers.
2. **Bengio et al., A Neural Probabilistic Language Model**
   The fixed-window neural language model your next-word lab is a miniature of.
3. **Jurafsky and Martin, Chapter 3: N-gram Language Models**
   Entropy, cross-entropy, and perplexity in the language-model setting.
4. **Goodfellow, Bengio, and Courville, Deep Learning, Chapter 6**
   A rigorous treatment of deep feedforward networks.
5. **Deep Learning, Chapter 4: Numerical Computation**
   Gradient-based optimization, numerical stability, and softmax details.
6. **Rumelhart, Hinton, and Williams, Learning representations by back-propagating errors**
   The classic backpropagation paper.
7. **Christopher Olah, Calculus on Computational Graphs**
   A visual intuition for backpropagation as local gradients moving through a graph.
8. **Matt Mazur, A Step by Step Backpropagation Example**
   The same by-hand arithmetic we did in the worksheet, worked on a different small network. A good way to check that the method, not the numbers, is what you learned.
9. **CS231n Neural Network Case Study**
   A readable from-scratch classifier with softmax loss and backpropagation.